

Thesis Report

Trustworthy AI-Enhanced DevSecOps: Bridging XAI and RAG for Proactive Vulnerability Mitigation and Adaptive Pipeline Optimization

by

Md Sayem Mottakee
21301080

Nowshin Reza
21301042

Ariful Islam
21301016

Md. Adnan Parvez
21301017

A thesis submitted to the Department of Computer Science and Engineering
in partial fulfillment of the requirements for the degree of
B.Sc. in Computer Science and Engineering

Department of Computer Science and Engineering
Brac University
January 2026

© 2026. Brac University
All rights reserved.

Declaration

It is hereby declared that

1. The thesis submitted is my/our own original work while completing degree at Brac University.
2. The thesis does not contain material previously published or written by a third party, except where this is appropriately cited through full and accurate referencing.
3. The thesis does not contain material which has been accepted, or submitted, for any other degree or diploma at a university or other institution.
4. We have acknowledged all main sources of help.

Student's Full Name & Signature:

Md Sayem Mottakee

21301080

Nowshin Reza

21301042

Ariful Islam

21301016

Md. Adnan Parvez

21301017

Approval

The thesis titled “Trustworthy AI-Enhanced DevSecOps: Bridging XAI and RAG for Proactive Vulnerability Mitigation and Adaptive Pipeline Optimization” submitted by

1. Md Sayem Mottakee (21301080)
2. Nowshin Reza (21301042)
3. Ariful Islam (21301016)
4. Md. Adnan Parvez (21301017)

of Fall, 2025 has been accepted as satisfactory in partial fulfillment of the requirement for the degree of B.Sc. in Computer Science on January 31, 2026.

Examining Committee:

Supervisor:
(Member)



Muhammad Iqbal Hossain, PhD

Associate Professor
Department of Computer Science and Engineering
Brac University

Co-Supervisor:
(Member)

Md. Aquib Azmain

Senior Lecturer
Department of Computer Science and Engineering
Brac University

Thesis Coordinator:
(Member)

Dr. Md. Golam Rabiul Alam

Professors
Department of Computer Science and Engineering
Brac University

Head of Department:
(Chair)

Dr. Sadia Hamid Kazi

Associate Professor and Chairperson
Department of Computer Science and Engineering
Brac University

Abstract

Modern software development faces many challenges in balancing rapid delivery with robust security. The challenges are bugs, vulnerabilities and inefficiencies. The AI-based DevSecOps tools that are being used today are effective at detecting abnormalities and vulnerabilities but they act as a black box that provides little to no transparency or context that leaves the security team a little skeptical and makes it hard for them to trust the tools and act on them. Based on this struggle, this paper introduces a solution, an AI-driven DevSecOps framework that uses Explainable Artificial Intelligence (XAI) which provides insights into the decisions made by the transformer model LogBert. This framework also uses Retrieval-Augmented Generation (RAG) that dynamically retrieves contextual data(i.e. Historical fixes, security protocols and optimization patterns). LogBert is trained to model normal system behaviour, detect anomalies while SHAP boosts interpretability and RAG produces historical context which fosters trust from developers and collaborative debugging to eliminate any doubt which is induced by the black box dilemma. In this framework security is actively strengthened through real-time vulnerability scanning, threat prediction and automated compliance check. Moreover, LLM is combined with SHAP to describe what the output of SHAP means, making it more human interpretable. Also it is applied to RAG which transforms retrieved contextual information into simple human readable texts and proposes fixes. This proposed framework does not only improve performance but it also provides a clear path for using AI in a way that is both sustainable and transparent within DevSecOps test phase. Keyword : DevSecOps, Trustworthy, AI, Proactive Vulnerability Mitigation, XAI, SHAP, RAG, Security Automation, Real-time Threat Detection, CI/CD, Transparent AI Systems **Keywords:** Trustworthy AI, DevSecOps, Explain-

able Artificial Intelligence (XAI), Retrieval-Augmented Generation (RAG), Large Language Model(LLM),Proactive Vulnerability Mitigation, Adaptive Pipeline Optimization, Security Automation, Real-time Threat Detection, CI/CD, Transparent AI Systems

Table of Contents

Declaration	i
Approval	ii
Abstract	iv
Table of Contents	v
List of Figures	vii
Nomenclature	viii
1 Introduction	1
1.1 Background	1
1.2 Rational of the Study or Motivation	2
1.3 Problem Statement	2
1.4 Objective	3
1.5 Methodology in Brief	4
1.6 Scopes and Challenges	6
2 Literature Review	9
2.1 Preliminaries	9
2.2 Review of Existing Research	10
2.3 Summary of Key Findings	19
3 Final Specifications, Requirements, and Impact Assessment	20
3.1 Final Specifications and Requirements	20
3.1.1 Functional Requirements	20
3.1.2 Non-Functional Requirements	21
3.2 Societal Impact	22
3.3 Safety Impact	22
3.4 Legal Impact	23
4 Proposed Methodology	24
4.1 Design Process or Methodology Overview	24
4.1.1 Problem Definition and Objective Formulation	25
4.1.2 Sub-Phases of DevSecOps Test Phase	26
4.2 Data Collection	26
4.2.1 Data Cleaning	27
4.2.2 Data Transformation	28

4.2.3	Data Reduction	30
4.2.4	Final Preprocessed Dataset	30
4.3	Model Architecture and Training	31
4.3.1	Model Architecture	31
4.3.2	Training Strategy	32
4.3.3	Evaluation	34
4.3.4	LogBERT-Based Anomaly Detection Results	36
4.4	SHAP- and LLM-Based Explainability	39
4.4.1	SHAP-Based Explainability	39
4.4.2	LLM-Enhanced SHAP Explainability (SHAP+LLM)	41
4.4.3	Practical DevSecOps Benefits (SHAP + LLM)	47
4.5	RAG- and LLM-Based Explainability	47
4.5.1	RAG-Based Context-Level Explainability	47
4.5.2	LLM-Enhanced RAG Explainability (RAG+LLM)	50
4.5.3	Practical DevSecOps Benefits (RAG + LLM)	56
5	Performance Evaluation and Analysis	57
5.1	Performance Evaluation	57
5.2	Validation of LogBERT Anomaly Detection	57
5.3	Analysis of Design Solutions	58
5.4	Comparisons and Relationships	59
5.5	Final Design Adjustment	60
5.6	Discussion	61
6	Conclusion	64
6.1	Summary of Findings	64
6.2	Contributions to the Field	65
6.3	Recommendations for Future Work	65
6.4	Final Remark	65
	Bibliography	67

List of Figures

1.1	Methodology Workflow	5
2.1	DevSecOps Pipeline	9
3.1	DevSecOps Test-Phase Explainability Pipeline	21
4.1	LogBERT architecture	32
4.2	LogBERT Training Performance by Epoch	35
4.3	Confusion matrix showing raw prediction counts	37
4.4	Evaluation Metrics	38
4.5	Token-level SHAP visualization across three log sequences.	42
4.6	Token-level SHAP explanation for a single log sequence.	42
4.7	SHAP + LLM explainability pipeline	43
4.8	RAG Explainability Workflow in DevSecOps Test Phase	49
4.9	RAG + LLM Explainability Workflow in DevSecOps Test Phase	52
5.1	End-to-end workflow of the system	61

Nomenclature

The list of symbols and abbreviations that will be used later in the document's body is provided below:

AI	Artificial Intelligence
XAI	Explainable Artificial Intelligence
RAG	Retrieval-Augmented Generation
DevSecOps	Development, Security, and Operations
CI/CD	Continuous Integration / Continuous Deployment
ML	Machine Learning
RL	Reinforcement Learning
SHAP	SHapley Additive exPlanations (XAI technique)
GAN	Generative Adversarial Network
MTTR	Mean Time To Respond
IaC	Infrastructure as Code
SAST	Static Application Security Testing
DAST	Dynamic Application Security Testing
SCA	Software Composition Analysis
OPA	Open Policy Agent
LLM	Large Language Model
MTTD	Mean Time To Detect
NLL	Negative Log-Likelihood

Chapter 1

Introduction

1.1 Background

DevSecOps has become essential in the dynamic world of software development where the incorporation of security processes in the software development iterative process known as DevOps is essential. DevSecOps focuses on integrating security throughout the lifecycle in the software development process so that the resilience of software and secure applications remain in check and safeguarded against system susceptibilities. With organizations adopting the objective of quicker and efficient delivery processes, DevSecOps helps to guarantee that security does not make the stage of an afterthought in continuous integration (CI) and continuous deployment (CD) processes but is rather, part and parcel of the process. Nonetheless, the typical approaches to addressing security in DevOps systems have a tendency of facing issues of scale, the amount of data and intricacies of contemporary forms of threats, resulting in lapses in weakness detection and fixing processes [19], [9].

Artificial intelligence (AI) has become an instrument to change the way DevSecOps processes can be optimized. By automating threat detection, vulnerability management and incident response, the AI will make the processes around the identification of security risks faster, minimizing human intervention and increasing the overall agility through development pipelines [8], [11]. Although these things can lead to integrating AI into DevSecOps, there are a few obstacles to that. Particularly, the lack of explanations of how AI models work, otherwise known as the black-box problem, makes it more complicated to comprehend AI-based decisions, and results in a lesser degree of trust that security teams associate with automated systems [15].

To address these concerns, transparency and explainability have been some of the focus points in developing AI in DevSecOps. Explainable AI (XAI) aims at increasing the explainability of AI models, such that the humans can understand the rationale of the decision made by an AI system. Such explanations are a primary consideration in high-stakes contexts where explanations of the decision made by automated security systems can augment trust and compliance with regulations [3], [19]. Moreover, methods such as Retrieval-Augmented Generation (RAG) are under discussion to increase the speed of real-time decisions made by replacing the history provided externally to the AI models, increasing security responses accuracy and context specificity to a situation.[16], [1].

The combination of XAI and RAG can provide a strong solution to the requirements of automated security as well as a need of security that is able to be trusted and adaptive to real-time solutions in the context of DevSecOps.

1.2 Rational of the Study or Motivation

DevSecOps tools with AI capabilities have been able to improve automation and security in software development pipelines to a large degree. But the main problem is that such tools usually have a fatal flaw because they lack transparency and explainability. Any AI-powered system currently present in DevSecOps, including the anomaly detection and vulnerability scanning processes, is in a form of a black-box model, wherein the workings of the AI remain unclear and do not enable security professionals to follow and essentially trust the AI-based recommendations properly [8], [9].

Combining Explainable AI (XAI) and Retrieval-Augmented Generation (RAG) technologies will have massive potentials in filling the gaps. XAI is capable of making the decisions of AI more understandable, thus building trust and making decisions in the areas of real-time security that seem more educated [19]. In the meantime, RAG can augment the decision-making capabilities by integrating outside sources of knowledge thereby providing more precise, situational awareness advice in regard to mitigating the threat [15]. The improvements of AI-driven DevSecOps frameworks that the research shall target entails the incorporation of XAI and RAG, thus enhancing the level of transparency and effectiveness of the security processes automated. That would introduce significant changes into the pattern of DevSecOps and allow ensuring greater reliability and interpretability of AI tools, eliminating one of the main drawbacks and improving the security stance of software delivery pipelines.

1.3 Problem Statement

The fast development of the AI technologies has brought a lot of changes to the DevSecOps domain, but certain issues remain when it comes to the transparency and the explanatory capabilities of the AI-based security devices. The current AI models tend to be of the black-box class and hence it is hard to trace a security professional to trust in the computer-made decisions. This ambiguity compromises the efficiency of the decision-making process and is a risk of compromising the integrity in security practices especially in high-velocity development environments [9]. Moreover, the lack of IT automation in real-time denies a quick response to new threats. These transparency gaps and automation gaps are drawbacks of wider adoption and reliability of AI in DevSecOps [2]. To overcome such challenges, solutions appear to lie in marrying both Explainable AI (XAI), which provides additional insights to make decisions that are both trustworthy and having an operational real-time and contextually aware use that generates a level of actionable decision-making using Retrieval-Augmented Generation (RAG) [9], [15].

1.4 Objective

The purpose of the study is to progress the field of anomaly analysis in DevSecOps test-phase pipelines, by creating a framework that makes interpretation in determining anomalous behavior patterns in execution logs. The method uses **logBERT** to learn balanced operational behavior using structured log sequences and detect abnormal behavior that is evidence of abnormal execution patterns. In order to interpret these deviations, **SHAP**-based token-level attribution and **Retrieval-Augmented Generation (RAG)** are combined to recall semantically similar historical log sequences. The resulting behavioral insights, the contextual evidence, and attribution outcomes are then synthesized into the form of structured explanations (readable by a human) and actionable remediation advice by a reasoning layer constructed using **LLM**. In this way, the paper offers understandable, contextual, and operationally feasible interpretation of anomalies, offering quality diagnostics and decision-making in **DevSecOps test pipelines**.

RQ1-Anomaly behaviour Detection

How can anomalous execution in DevSecOps pipeline (test phase) can be identified from log sequences by modeling deviation from normal system logs?

RQ2-Unified Token-Level and Contextual Explainability

What is the best way to combine token-level explanations based on SHAP and historical log context retrieved by RAG to give clear and interpretable explanations of identified DevSecOps anomalies?

RQ3-Evidence-Based Explanation and Generation of Actions

In what ways might an LLM-based reasoning layer map the causes of anomalies at the token-level and the historical evidence retrieved into compact and human readable form and actionable remediation proposals concerning anomalous DevSecOps behavior?

Summary of How the Research Addresses These Questions: The framework answers these research questions in a three stage integrated pipeline. To begin with, LogBERT models a regular DevSecOps execution behaviour and detects the anomalous behaviour by noting the deviation in the likelihoods of log sequences. Second, SHAP quantifies how the few tokens in the form of logs influence such deviations and RAG identifies semantically similar historical sequences so that they can be put into a contextual background. The combination of the local causality and the global historical evidence on this dual explanation mechanism provides a more complete explanation as compared to the previous method. Lastly, the LLM reasoning layer takes results of attribution and context and produces logical, human-understandable explanations and recommended remedial activities. The behavioral deviation model, dual SHAP+RAG explication and operationally actionable action by the LLM is a new combination that offers an interpretable and operationally actionable framework, overcoming the weaknesses of the current DevSecOps log analysis.

1.5 Methodology in Brief

The work investigates how to add AI-powered techniques, namely Explainable AI (XAI) and Retrieval-Augmented Generation (RAG), to the DevSecOps pipeline to make the test phase activities more visible and automatable. The given approach is mainly experimental in nature, and it is believed that the design, development, and testing of an AI-based framework that accommodates real-time security monitoring and provision of overall optimization in DevSecOps pipelines will be achieved. The overall methodology is shown in Figure 1.1.

This study will start with data harvesting of artificial CI/CD pipelines, including real threats, security logs, intelligence feed, and continuous deployment information. This information is used as training data on deep learning model that are optimized to identify possible vulnerabilities, future threats, and automate a response. No special data sets will be used; existing datasets including security logs of GitLab CI, Jenkins, and threat intelligence platforms will be used.

Such method as SHAP as XAI techniques will be implemented so that the model can be interpreted, and security teams gain transparent and actionable insights about AI-based decisions. By incorporating the RAG models, the system will be able to issue context-sensitive recommendations in real-time by supplementing the decision-making process in security and large-scale knowledge retrieval.

The performance as the means to justify the performance of the framework will be considered based on a predetermined audience: accuracy rates in revealing the vulnerabilities, false positive rate drop and the increase of the mean time to respond (MTTR) to the security the incidents. Moreover, the comparison of the AI-based tool will be conducted with the previous security measures in DevSecOps pipelines.

The last test will be conducted to check developed models against both known security threats and zero-day simulations with real world attacks scenarios generated on special dataset of synthetic Generative adversarial networks (GANs).

The given methodology will show how AI can be integrated in the processes of DevSecOps to improve the security, agility, and scalability of software delivery systems without sacrificing transparency and trust in explainable decision-making in action.

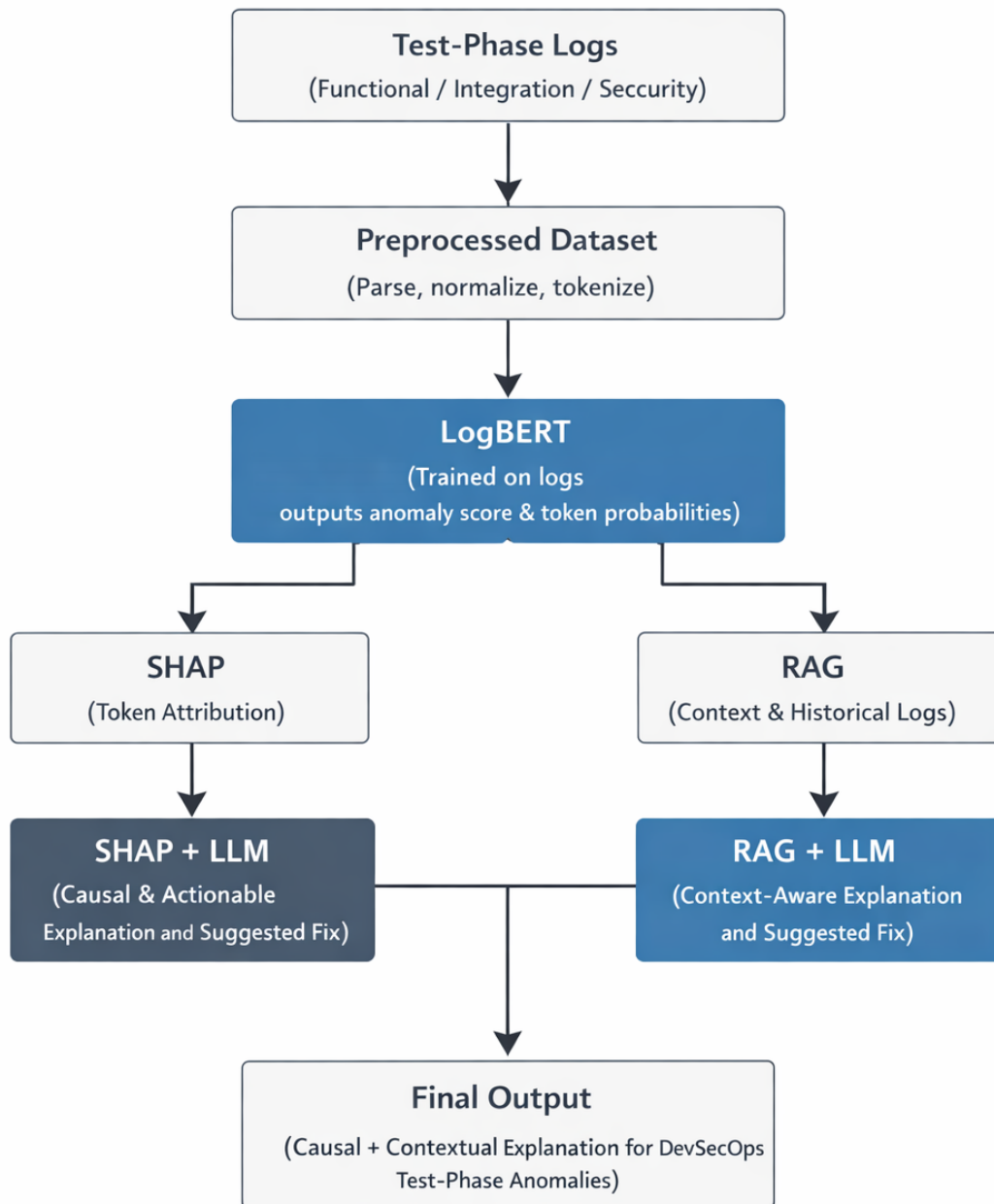


Figure 1.1: Methodology Workflow

Methodology Workflow for AI-Driven DevSecOps Optimization

1. Data Collection

- **Description:** Gather security logs, telemetry data, threat intelligence feeds, and CI/CD pipeline data.
- **Key Sources:** GitLab CI logs, Jenkins security logs, real-time monitoring data.

2. Deep Learning Model: LogBERT

- **Description:** Train the LogBERT deep learning model to analyze DevSecOps logs, assign token-level anomaly scores, and support interpretable anomaly assessment.
- **Key Techniques:** Transformer-based sequence modeling, token-level scoring, SHAP for explainability.

3. XAI Techniques (Explainable AI)

- **Description:** Apply XAI technique, SHAP, to ensure transparency in the decision-making process of AI models.
- **Key Purpose:** Explainable results that can be interpreted by security teams.

4. RAG Models (Retrieval-Augmented Generation)

- **Description:** Integrate RAG models to enhance real-time decision-making by combining large-scale knowledge retrieval and generative AI.
- **Key Techniques:** Retrieval-augmented responses, adaptive feedback for automation.

5. Automation and Optimization

- **Description:** Automate security decisions and responses in the pipeline, enhancing efficiency and minimizing manual intervention.
- **Key Tools:** Anomaly response automation, self-healing security models.

6. Testing and Evaluation

- **Description:** Test the developed models and the integrated DevSecOps framework against known and zero-day threats to evaluate effectiveness.
- **Key Metrics:** Accuracy, reduction in false positives, Mean Time To Respond (MTTR), performance improvements.

1.6 Scopes and Challenges

Scopes:

- **Real Time Process Enhancement:** The purpose of the study is to specifically specialize in the area of optimal DevSecOps pipeline by applying AI technologies to the specifics of Explainable AI (XAI) and Retrieval-Augmented Generation (RAG) to improve strength and visibility in the test automation, threat detection and vulnerability management of continuous integration and deployment (CI/CD).

- **Transparency and Automation:** One of the key spheres where the research will be implemented is the introduction of XAI to ensure that the DevSecOps activities that are executed by automated machines are readable and understandable. This enables testing teams to have faith in the ruling of the system which is essential to the implementation of automated testing measures in sensitive and high-stakes conditions as noted by multiple automated testing strategies of DevSecOps using AI [6].
- **Real-Time Threat Detection and Remediation:** The goal of the research is the enhancement of the real-time monitoring ability of the DevSecOps pipelines. The system is preconditioned to adjust to new challenges and respond to them in an automated and real-time way adapting to unseen risks using reinforcement learning and other AI models. This especially applies to the fact that the use of AI-based instruments greatly enhances the speed at which real-time security systems deliver responses to threats [5].
- **Limitation to Specific DevSecOps Phases:** This study is focused on optimizing certain stages of the DevSecOps lifecycle, and to a greater extent, these are testing phases (e.g., test automation, fault detection, quality assessment) and test-driven feedback loops. It does not cover the complete software development lifecycle but aims to enhance the effectiveness of testing in the CI/CD pipeline.

Challenges:

- **Integration Complexity:** Application of AI models (including such models as XAI and RAG) within the established DevSecOps processes is hardly simple. A lot of DevSecOps tools were created, but they lack AI features, so to use them effectively and without obstacles, one will have to address the challenges of tool compatibility and merge them with the current systems. Multifacetedness of embedding the models based on AI into the already existing DevSecOps tools has been singled out by various researchers as the vital obstacle. [8]
- **Data Quality and Availability:** The work of AI-based systems depends on high-quality data and its availability. Logs, alerts and telemetry with incomplete or non-congruent data may decrease the effectiveness of the AI models and can result in inaccurate prediction and increase false positive as some even have it fixed in different AI-based threat detection pipeline. [19]
- **Model Interpretability:** Nevertheless, the vision of XAI has yet to fully achieve complete interpretability, even in sophisticated models. Without a clear understanding of why the AI makes certain decisions, testing teams may struggle to trust the information provided, which can hinder the adoption of AI-driven testing systems. This challenge is frequently acknowledged in research aimed at increasing AI transparency to improve trust and acceptance in automated testing environments. [16]

- **Scalability and Computational Costs:** Since deep learning models as well as reinforcement learning systems consume much of the computational resources, it is a difficult task to scale them to function in huge and distributed DevSecOps settings. This will act as restriction to their application in real life scenarios where resources are comparatively scarce which has been the bane of the research in AI scaling[5].
- **Opposing and Rising Threats:** Adversarial attacks are not a recent occurrence in AI models, in which inputs may be used to achieve the wrong or deceptive output by a malicious individual. Moreover, due to the changing nature of software systems and testing needs, there is need to keep AI models updated and retrained with new testing data, and thus real-time validation of test results and reliability remains a recurrent problem. The problem of adversarial vulnerabilities in AI models in the DevSecOps testing framework has been discussed in a number of research papers. [11].
- **Regulatory and Ethical Issues:** The prospect of application of AI in testing creates crucial questions of ethical and regulatory dangers. A key challenge is ensuring adherence to privacy regulations (e.g., GDPR) when sensitive data is processed within DevSecOps testing pipelines. Additionally, it is crucial to ensure that AI-based models are fair and unbiased in their decision-making to prevent unintended outcomes during testing. Several studies emphasize the need to balance privacy concerns with the adoption of AI in DevSecOps testing environments [8].

Chapter 2

Literature Review

2.1 Preliminaries

DevSecOps uses the model of integrating security in each phase of software development process life cycle. It maintains consistent security through automation of codes, testing, development and deployment. DevSecOps gives an opportunity to development, security and operations teams to collaborate and find a way of reaching high velocity and secure software deployment. The proposed system architecture is illustrated in Figure 2.1.

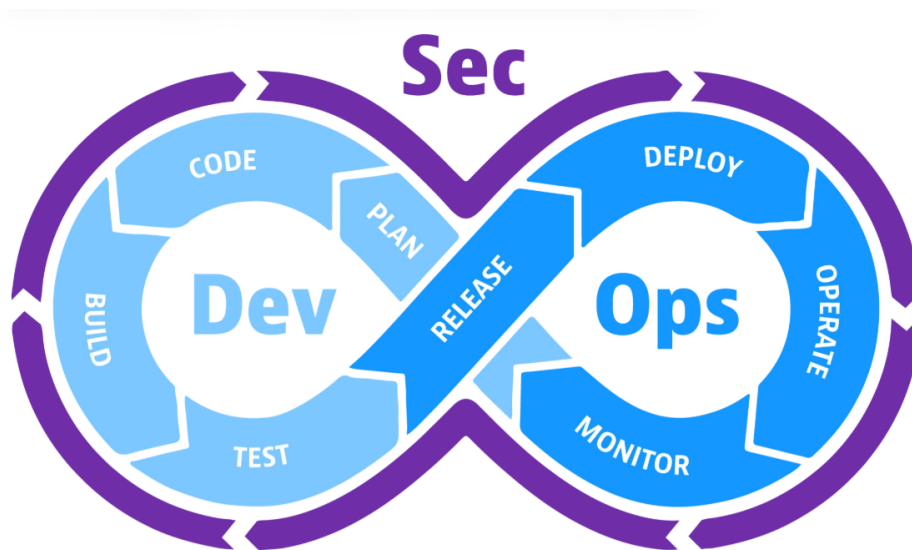


Figure 2.1: DevSecOps Pipeline

Artificial Intelligence (AI) is another tool involved in DevSecOps because it enhances automation in most activities such as vulnerability checks, incidents detection, and types of anomalies. What Machine Learning (ML) models predict is the risk, and the manner in which that risk can be streamlined through its past history. However, AI models may be opaque and the model decisions are likely to be Inexplicable and individuals cannot have confidence in it.

Explainable Artificial Intelligence (XAI) is utilized as a counter against AI

opacity to make a model operate on intelligible and understandable decisions to its human users. To know the features that influenced the output of a model, other methods like SHAP (SHapley Additive exPlanation) and LIME (Local Interpretable Model-Agnostic Explanations) are applicable. The easiest XAI technology to apply in DevSecOps is the spot checking of the discovered vulnerabilities, prediction explanation, and teamwork debugging.

Retrieval-Augmented Generation (RAG) consists in mixing, but not in mixing any generative language model and any document retrieval system. It allows the model to query any form of context like searching of older security incidents or compliance standards or patch versions so that quality and informed results are obtained. RAG in DevSecOp enables the models to search the latest threats, knowledge or historical remediation that would assist to refine the code and respond to the incidents.

CI/CD Pipelines (Continuous Integration/Continuous Deployment) are key components to DevSecOps that have some processes that can be coded using AI-based applications including quality checks, vulnerability scan analysis, and approval of a deployment process. RAG would enable this by mining the best practices as far as configuration is concerned, and the XAI would not only assist the developers, but also the security departments by notifying about the decisions that were taken with the assistance of AI.

Threat Intelligence is the researched one on the cyber threats that are utilized to stop and counter the attacks. Real time evaluated risk can be achieved using RAG enabled systems, and they can perform automatic response since they access the most recent information in relation to new vulnerabilities, malware trends, and attack vectors. This kind of hook allows a dynamic adaptation, and it does not require strict rules.

Compliance Automation is enhanced by The AI models that detect violations of policies of the code and propose their fixes. The models provide relevant compliance rules through RAG and can be used to assist enforcement measures before engineers or auditors.

2.2 Review of Existing Research

With the implementation of the DevSecOps concept, Pandya [12] notes that the model of compliance in AI-oriented terms is being developed with the potential participation of constructs like smart contracts and LLM that would automate software compliance under the scenario of continuous integration and continuous deployment (CI/CD) pipelines. Pandya [12] makes assumptions about the future of this model, approaching it as a question of automating software compliance in this or that concrete scenario of the CI/CD pipelines. The study will focus on the sophisticated part and human labour applied in checking compliance of the healthcare business which encompasses the HIPAA field. This model introduces the highly advanced LLMs to use the dynamic-regulatory documents that generate smart contracts that will receive operations within the networks of the special blockchains with an inbuilt point of compliance and an extended trace which has no possibility to be altered.

Retrieval-Augmented Generation (RAG) is intra-document world models that desensitize and acclimatize LLM to give it context-palat seasoned generations having in real-time lucrative compliance information. A comparative analysis indicates higher efficiency and accuracy as opposed to established methods of manual compliance. Despite the promises of innovation, several challenges exist to the process of incorporating LLMs and blockchain, including scalability problems, complexity of incorporating different CI/CD, and the necessity to track the compliance data within the highly accelerated development. Moreover, it is still necessary to have human expertise in regulation interpretation in unclear regulatory settings. The further plans involve the extension of the flexibility of the compliance automation in a variety of industries, the use of prototypes to perform empirical assessment, and the enhanced adaptability of AI-based models. Regardless of the possible concerns, this paper demonstrates that AI potentially can transform the field of regulatory auditing within the DevSecOps process entirely, provided that one copes with important barriers to implementation.

Ray [18] introduces the concept of vibe coding, which composes software based on natural language instructions with the use of AI tools, such as large language models (LLMs), and retrieval-augmented generation (RAG). The aim of this approach is to promote a collaboration between AI and humans, raise programmer efficiency, and simplify programming. Some of the significant DevSecOps impairments that it can automate include code creation, deployment, and security checks. As an example of context-aware policy enforcement and coding, Open Policy Agent (OPA) and Zencoder AI are cited. However, security vulnerability, degradation of the skills of the developers, and AI errors are dangers. To sustain the same openness level and confidence, the study highlights how explainable AI (XAI) will be required. The next research should be structured on the topic of security governance strengthening, better interpretability, and tool synchronization with teamwork.

The development of a revolutionary CI/CD system SecureAI-Flow is suggested by Rahman and Biplob [17] to handle the security needs of AI applications in DevSecOps. To accommodate the statistical code analysis, model soundness validation, safe containerization, as well as dynamic threat surveillance throughout the AI development across the lifecycle, this framework integrates autonomous agents. The principal innovation is the explainable AI techniques (XAI) such as SHAP and LIME are used by the security agents such as CodeGuard-AI in which automated security decisions can be explained and trusted by human users. Also, the system incorporates RAG-based processes to obtain periodic compliance changes and adaptive threat identification. Its primary contribution is the use of smart contracts for tamper-impervious audit trails and real-time compliance monitoring, which is very useful in an MLOps environment. These advantages notwithstanding are some of the main disadvantages of SecureAI-Flow, such as the computational cost associated with running agents in real-time, the potential high false-positive and false-negative rates of categorizing threats, and failure to deploy the technology in production environments based on empirical findings. The authors recommend that future research focus on multi-party and multi-agent collaboration, as well as compliance with the EU AI legislation, in order to scale the framework to industry-level DevSecOps platforms. Although there is still a problem of practical use and performance opti-

mization of which to investigate in the future, SecureAI-Flow that mixes together the concepts of real-time automation, transparency with flexibility is on the whole an example of proactive factors to protect AI pipelines.

To examine the ways AI and machine learning can be applied into DevOps with the purpose of enhanced automated software deployment and maintenance, Oyeniran et al. **Oyeniran2023AI** examine the current state of the art in deploying and maintaining software in automated processes. Data science models such as TensorFlow and PyTorch are applied together with observability tools such as Datadog and New Relic to incorporate AI into key pipeline features such as code analysis, anomaly detection and predictive analytics. CI/CD automation is further achieved by Jenkins, Docker, and Kubernetes. Self-healing systems that are possible through Google Cloud and AWS Auto Scaling make systems more reliable and reduce downtimes. The significant challenges are data quality, complexity of integration and the ethical problem with AI bias and transparency. Although an address is not given on XAI, the interpretability necessity is present in its absence. Areas where opportunities lie in the future include federated learning, full automatization of DevOps and applying quantum enhanced computing to allow safe scale-up.

The research [5] explores how Artificial Intelligence (AI) can support a security posture of the DevSecOps pipelines by automating some of the security processes. The authors focused on using AI in CI/CD lifecycle to find vulnerabilities, minimize false positives, and increase efficient real-time policy enforcement. AI tools especially increased the quality of static, dynamic and interactive application security testing (SAST, DAST, IAST), leading to a notable positive change in response to anomalies and security incidents. The paper does not explicitly refer to Explainable AI (XAI) and Retrieval-Augmented Generation (RAG), however, the idea of the need of interpretation of the results of decisions and their real-time production complies with their postulates. Namely, the fact that implementing AI could majorly decrease the cost of security testing because it can contribute to the discovery of the real threats and the security of shorter delivery cycles is one of the contributing factors as well. However, challenges such as obscurity of AI models, the challenge of integration and susceptibility to malicious intrusion arise. The report proposes to conduct further studies about explainability, model resistance to adversaries and continuous training in response to evolving threats. On the whole, the article makes a point of the role of AI in automating and enhancing the software security testing within the framework of DevSecOps and further confirms the significance of AI in terms of the current security architecture, although its application can still be optimized as far as interpretability and easiness of implementation is concerned.

The study shall focus on the application of AI in DevSecOps so as to identify the threats, measure the threats and prevent the specific threats [19]. It recalls the importance of machine learning and natural language processing (NLP) to increase the degree of real-time detection of danger and cut human efforts. The concentration of explainable AI (XAI) is the openness and the confidence as the reinforcement learning takes responsibility in the adaptive response. Either or one of the said methods such as decision trees, SVMs, deep learning, clustering and autoencoders are employed to detect anomalies. Threat intelligence technologies based on artificial

intelligence minimize false positivity and improve proactive defense. However, it is also flawed, including the dependence on quality of data, vulnerability to malevolent attacks, and difficulty of integration. To generate even more trust and scalability regarding artificial intelligence (AI)- powered DevSecOps, as suggested by the article, the potential future paths can be proposed by federated learning, the highly robust AI models and explainability.

This literature assessment of AI-based security solutions to DevSecOps is completely discussed in Paper [15]. It compares threat detection model, compliance model, and shift-left, among which are Random Forest, LSTM, DBN, CNN, and MLP. The important aspect of transparency and real-time responsiveness is also highlighted in the research, regardless of the lack of the direct address of these concepts, endorse the utilization of XAI and RAG. Such tools as CyberDevOps, DevPrivOps, and 6Detector, are the ones we mention. One of the key discoveries is that the CI / CD systems must be scalable and light weight. Here are such issues as high false positive rates and lack of empirical data that are discussed in the paper as well. To grow in confidence and acceptance, in further research, specific priority should be paid to the clarity, privacy-preserving architecture, and testing in the actual multi-cloud setting.

The very one-encompassing strategy applied in the paper [10] incorporates the intelligence of threats in the whole delivery process of software, as well as automation of security procedures by means of implementation of AIOps (Artificial Intelligence for IT Operations) in DevSecOps. This study lays much emphasis on proactive security automation via the deployment of AI anomaly detection, proactive predictive analytics as well as intelligent incident response mechanisms. Other than the CI/CD platforms (GitLab and CircleCI), now we have such tools as Moogsoft, Splunk, Trivy or even TensorFlow. The paper also brings out the synergy between AIOps and DevSecOps that would help build the resiliency of operations and minimize the breach levels by incorporating continuous threat identification as a part of the development pipelines. Despite the fact that RAG-like concepts find a good reflection in the application of AI-powered threat intelligence, they explain that in XAI, it is cited as an essential process that enables their explainability and traceability in terms of automated decision-making. The authors outline three key problems that can smack the potential of the framework, namely, bad acquisitions in terms of the quality of acquired information, reluctance towards the adoption of AI in companies, and regulatory compliance hiccups. To break the existing boundaries, they suggest that it is worth looking into federated learning, self-evolving threat models and autonomous cyber-defense agents further. The research would offer an outstanding architectural design of AIOps-enabled DevSecOps; in so doing, the governance and explainability of AI will be encouraged to increase the security automated systems, triggers, and speed the maturation of the development of the security-integrated software-building behaviors.

Michael Fu et al. [8] is a detailed examination of the 99 papers (2017-2023) dealing with deploying AI to enhance security in DevSecOps. The 12 examples of those sufficiently meaningful areas in which the AI-based tools of machine learning and deep learning can most effectively be used are listed in this paper: vulnerability detection, anomaly detection, and dependency analysis among others. It dwells

on the explainability of the explainable AI (XAI) regarding building trust with the granular models applied by the security engineers. The paper also explores retrieval-augmented generation or RAG in order to enhance the capability of the big language models in providing security recommendations based on the content. Discussion of performances of such tools as Snyk, Infield, and DeepPerf is also there. They include low transparency, cumbersome feature engineering and poor model generalization, among others. New studies should enhance RAG-based systems and XAI methods to shatter demands of scalable, reliable, and proactive measures of DevSecOps security.

As mentioned in the works by Hussain and Shabir [9], the authors explore the effect of AI implementation on DevSecOps and prove that despite the substantial improvement of the security state already realized with the help of automatization of threats detection, vulnerability search, and patching, it is still possible to improve it significantly. The paper outlines how predictive DevSecOps strategies may be achieved by using AI-based technologies to predict the occurrence of threats, optimize resources, and help with mitigating risks using the proactive approach. The necessity of future transparency in AI-based decision-making was mentioned in the paper in the frame of behavioral analysis and real-time scanning, and it becomes obvious that it is possible to use Explainable AI (XAI) and Retrieval-Augmented Generation (RAG). The challenges that have been cited include data privacy issues related to data ownership and control, moral concerns of bias in algorithms, as well as challenges in corporate culture. What makes it vital is that, this article introduces developing more efficient AI algorithms, the inclusion of AI algorithms into the cloud-native and IoT systems and developing moral control regulatory bodies. The objective of our thesis to use explainable AI-assisted strategies to create safe, rapid, and steady DevSecOps are tightly knit to such factors.

This is a complete DevSecOps structure offered by Clarisse Feio et al. [7], and it is based on Continuous Security Testing (CST). It provides a managed pipeline, where it provides security tests at every phase of the CI/CD pipelines. This paper is special due to the broad mapping of the security operations to some parts of the development (e.g. SAST during the building, DAST during testing, and SCA during packing). The experimental approach was experimented on the case study which is experimented on real life basis and has been quite tolerable to the developers as many vulnerabilities were turned up in the development life. The very principles of proactive mitigation, which we exposed in our thesis are in line with the fact that this study focuses on the so-called shift-left security, i.e. the security enacted at the very beginning of the SDLC. It is decided to use Sonarqube, OWASP ZAP, and Dependency-Check to automate the tests, and the research has identified some major concerns, including the lack of the production deployment in the test environment, the lack of the adequate compatibility of the tools, and poor security culture among developers. Future work will entail a creation of new assessment products to establish the effectiveness and acceptability of CST tools. The empirical research can tell a lot about the real use of DevSecOps and it also tells about the importance of automation, tool integration and preemption of security which are the main components of our suggested AI-integrated solution.

The study by Dwivedi et al. [3] provides a detailed guideline of the explainable

Artificial Intelligence (XAI) in relation to its concepts, taxonomies and implementation. This work shall be employed in any practice that poses risks on the usage of AI on security-related practices like DevSecOps. Following the feature-based and instance-based interpretation, the paper reviews such methods as SHAP, LIME, PDP, Anchors and compares the global and local interpretability of it. Importantly, it raises a new potential of neuro-symbolic artificial intelligence with the capability of enabling causal-reasoning in an opaque model. Among the issues that have been addressed in the paper, one can also mention the price of computational loads of XAI methods and the fact that such a method can take advantage of the burden of interpretability rather than the complexity of the model, including the bias in the data that can be intensified by even the most transparent models. It also investigates the way XAI behaves in a distributed environment (when it comes to federated learning and AutoML). Possible future research directions would be creation of real time explanation tools and scalable symbolic reasoning tools. In particular, there is also the aspect of trust in Trustworthy AI and therefore this poll is so relevant to our theory. The XAI approaches as suggested and presented in the current paper introduce an approach that develops a methodological view as to how the transparent explainable models and auditable models would be developed and explained by using the Security testing and monitoring process via the DevSecOps pipelines and how the end-user/consumer shall be able to comprehend, debug the developed model and feel confident about the AI-system decisions that is driving them.

Charlotte [16], the author, reflects on the application of Machine Learning (ML) to automate the security of cloud-native pipelines in DevSecOps in such areas as threats, compliance, and resilience of systems. The paper shall describe how in real-time security ML models, supervised learning, unsupervised learning, and deep learning will help improve security by lowering mean time to detect/respond (MTTD/MTTR) and the ability to support the development of predictive risk assessment. It discusses as well how the privacy of training has been assured by resorting to federated learning in a distributed data source. The explainable AI (XAI) is proposed as a solution in response to the increasing demand of transparency and audibility (particularly in the regulated sectors). The relevance of edge security highlights the possibility of machine learning at a network edge to provide a low-latency response to the threat in the sphere of financial and healthcare industries. Despite the merits that the paper sustains, it also solves the problem of the model drift, hard combination, and interpretability. It is convinced in further popularization of self-tuning ML models and privacy-preserving frameworks. Our theory can contribute much to the current paper as it is focused on the applicability of transparency (XAI) and automation to DevSecOps, especially in its remote environments, most of which are cloud-based environments since adaptive learning and trusted environments are crucial to the regulation of security.

Olasehinde et al. [11] specifically highlight the importance of artificial intelligence in operational downtime of the real-time monitoring and the root cause analysis in the DevOps pipelines to process the said downtimes in the manner of enhancing efficiency and cost-effectiveness of responses. The article devotes itself to the use of the AI-based observability systems with the machine learning algorithms on the predictive analytics and frameworks regarding the anomaly detection. By automatically

decreasing or increasing the levels of the alerts, these particular systems automate responses of an issue thus introducing a considerable enhancement of the mean time to detect it (MTTD) and the mean time to resolve (MTTR). This has been implied because XAI itself is not explicitly stated but the necessity to be able to find root causes of the disease through the diagnostics program supports the implication of the significance of explainable models. The problems of model drift, poor quality telemetry data along with integration problems are thoroughly researched. In the report, scalable, explainable monitoring schemes and improvements in data piping are proposed. The reflection of our reasoning is quite evident since, at the monitoring phase of the DevSecOps, smart observability, and automatized diagnostics are supposed to be accurate and traceable as well. The argument in the research is presented in the favor of implementing complex CI/CD with adaptive and AI-based systems that suggest the essence of making the systems visible and adapt themselves to the ongoing operational factors.

Sandu [1] analyzes in detail how security may be made available throughout the DevOps process to make a system more resilient. The article is based on the way that cultural and procedural reforms are required in order to implement the security earlier during the software development cycles as well as use automated programs and resistance testing. Among its primary priorities, it is necessary to outline the automation of security based on Static and Dynamic Application Security Testing (SAST/DAST), GitLab, Jenkins, Docker, and Kubernetes. Importantly, Infrastructure as Code (IaC) and Security Orchestration, Automation, and Response (SOAR) systems allows one to react to the incidents quicker and ensure the sustainability of infrastructure compliance. XAI and RAG are not mentioned directly, but the fact that the paper stresses transparency, automation, and real-time threats detection falls into their activity. The most important issues include publicity of the organization, complexity of integrating and regulatory compliance. The article requires further research in DevSecOps supported by AI, in which DevSecOps is tested in real-time in order to fight threats and safe cloud-native-based architectures. This publication closely complements our thesis framework, not only in security integration in the early phases of product development, flexibility in automation, and cross-functional assimilation, which are all critical to the design of a dependable DevSecOps pipeline enabled by Artificial Intelligence.

Camacho [6] also pays attention to the question of how important it is to incorporate the concept of the AI and machine learning into the DevSecOps in order to automate the cycle of CI/CD and to address the vulnerabilities. It belongs to the research that is devoted to resource optimisation, predictive analytics and self-corrective with the involvement of the following tools as Jenkins, Docker, AutoML. Through the security and transparency report on Ethical AI, it is apparent the automation must be explained and interpretable although XAI and RAG has not been defined explicitly. Amongst the barriers, one can list a lack of experience on the part of practitioners, data security risk, and model murkiness. The article suggests that the further research to be done is likened to scalable evolution of machine learning pipelines and models. We build a hypothesis on DevSecOps, reliable, and flexible AI in testing and monitoring, and provide the example of the case study on how AI may contribute to the process of the removal of some human efforts, making it more

accurate and efficient.

Cankar et al. [2] propose a mixed approach where the combination of the machine learning-augmented static and dynamic testing tools in securing DevSecOps pipelines is involved. There are two primary tools that are described in the paper, LOMOS, an artificial intelligence-powered runtime log anomaly detection tool that uses natural language processing and deep learning, and IaC Scan Runner, which performs a static analysis of Infrastructure-as-Code (IaC). These tools give comprehensive protection coverage, which is dealing with design-time, as well as the run-time, of the pipeline. The study also notes the complementarity of SAST and DAST methods being carried out with machine learning to profile behaviors and identify anomalies. Although there is no explicit mention of XAI and RAG by the LOMOS, its log-based explanation of anomalies suggests requirements involving interpretability. Complexity of integration, capability of scaling AI/ML tools and keeping up to date security checks of IaC components are some issues that have been discovered. Future work The future work will be to better platform engineering and integrate these tools with SIEM systems for real-time threat response. This paper argues in favor of the trend to allow you to add the use of AI-enhanced monitoring and verification of security as seamlessly into the CI/CD pipeline, in line with the need to have flexible, trustworthy, and explainable automation of security in current DevSecOps systems, and underlines the trend of our thesis.

An issue, in which AI can be applied to the CI/CD pipeline, is given by Polinati [13] who suggests that the selection of the tests in them, the scheduling of builds, and failure prediction may be automated. The multi-agent systems and reinforcement learning are deployed to reduce the number of errors made through the scaling up and speeding up of deployment anomaly detection; there was reduction of 33 percent in building time and a 40 percent fastening up the speed by which the failure was indicated. A very narrow reference is made to explainable AI (XAI), the role of which with respect to transparency and trust is appreciated. An indirect implication of the research on the data-driven and adaptive optimization is that it does not refute retrieval-augmented generation (RAG). The most significant of them is scalability of the paradigm and integration complexity and ethical issues. The most relevant directions to our study and which falls into the scopes of intelligent, agile, and transparent DevSecOps monitoring and testing include federated learning, the AI-co-pilots, and the RL-based self-healing pipelines.

Yaddala, and Sunkara [14] provide an elaborate account on the web security vulnerability; both the ones that have been known and yet to be launched in the light of the emerging trends in the modus operandi of cyber-attacks in the year 2024. The categorization of the threats in the research is conducted according to the application; network, the levels of data privacy and the identification of the defensive and attacking roles that AI can perform. Other than facilitating the effectiveness of the more advanced forms of attack such as phishing that is in the format of AI and polymorphic malware, AI has demonstrated to stimulate detections of abnormalities, warning threats, and real-time response measures. The document is a close representation of the concepts of DevSecOps, which is the importance of automation, proactive protection, and monitoring aided by AI though a clear explanation of the

topic is not discussed. Requesting to make it more transparent and interpretable of models implemented in AI security, even in the hostile situation, is a tacit approval of XAI. Taking into account this growing risk of quantum computing that can kill the current encryption regime, the authors dispute the creation of quantum-resistant algorithms. High possibility of cloud services and uneven IoT security are also available. Research the future according to the explanations presented in the article is devoted to explainability, adversarial robustness, and the enhancement of the power of the AI tools in even more complicated systems that are networked. This piece of work just emphasizes the fact that the modern security systems should be equipped with intelligent and dynamic, transparent security mechanisms.

DevSecOps may involve the use of AI/ML as Pakalapati et al. [4] remark, which would make the system more resistant, automate the security tasks, and allow detecting threats in real-time. The study asserts that machine learning is the part of the automated incident response, predictive analytics and proactive anomaly detection. In order to examine the codes continuously, and to ensure a safe way of handling the credentials, such devices as OWASP, SonarQube, Fortify, and HIM-SCOrpsigns are added. There is the need to mention Explainable AI (XAI) in order to contribute auditability and transparency in respect to security-critical systems. The report suggests future research in the form of interpretable detection of threats, privacy-preserving and mitigation of biased techniques. On the whole, the researchers will be able to present evidence that can help in changing the DevSecOps security not only to be proactive, but no longer reactive.

The work of Pakalapati et al. [4] provides an elaborate technique to integrate AI/ML with DevSecOps to build its automation, security, and resilience. Considering the fact that threats are becoming more elaborate than ever before, the research aggravates the fact that powerful systems are becoming more of a necessity than it was before, further adding to the argument of paradigm shift wherein AI/ML becomes the vanguard of DevSecOps processes. It emphasizes the significance of such techniques as automated incident response, anomaly detection, and predictive analytics in preventing threats before their takeover. The inclusive DevSecOps adoption structure is also one of the most significant contributions of the paper since it addresses key steps such as feedback loops, ambassador recruitment, and stakeholder alignment. The effective application is examined through the use of such tools as HashiCorp Vault, SonarQube, Fortify and OWASP. The emphasis on trustworthiness in our thesis is considered by the issues raised about model interpretability and ethical AI adoption, although it does not include the concept of Explainable AI (XAI) or RAG. The paper has also provided performance metrics to monitor the success of adoption, and these include MTTA, MTTR, and the deployment rate. Summing it all, this research contributes to the development of our thesis greatly, giving both operational and strategic recommendations on how to integrate intelligent, safe, and adaptable mechanisms into DevSecOps environments and serving as one of the working guides to a successful integration of AI on a larger scale.

2.3 Summary of Key Findings

The provided works cover the study concerning the role of Artificial Intelligence (AI) and Machine Learning (ML) in DevSecOps by explaining the potential of automating the threats detection, easing the compliance process, and making the systems resilient. There is a vast amount of papers suggesting that the frameworks of the implementation of the AI models, i.e., LSTM, Random Forest, and deep neural networks, are included into the CI/CD pipelines to trigger the processes of anomaly detection, scanning the vulnerabilities, and enforcing the policies automatically. The tools that the AI observability and self-healing systems were complemented with most usually included Jenkins, Docker, SonarQube, and HashiCorp Vault among others. Among the solutions that stood out as especially relevant in high-stakes problem environments, namely that of the MLOps-related projects and regulated areas, explaining AI (XAI) approaches (e.g., SHAP) stood out as especially relevant when it comes to actively improving the accountability and interpretability of the models, and hence, their trust levels in those decisions.

To achieve security in both the design and execution processes, hybrid approaches that involve either static or dynamic analysis or log monitoring-based on NLP were proposed by some studies. Still in terms of compliance-driven investigations, the blockchain and smart contracts were explored to achieve the effect of an audit and tamper-proof monitoring. Despite the expressed concerns mentioned, Vibe coding and AI-assisted coding-enabled bots are more conciliatory and do not need much human interaction, even with their hallucinated results and over-dependency. The primary target of the presented study is the importance of early security deployment and, associated with it, the process of adaptive operations and threats on the go. Federated learning and reinforcement learning can be considered two of the most discussed concepts that are essential in preemptive alleviation.

There is some research that has suggested hybrid solutions whereby log monitoring coupled with NLP is combined with either static/dynamic analysis to guarantee security at both design as well as at run time. The blockchain and Smart contracts were also exploited to obtain an audit and tamper-proof watchdog effect in investigations that focused on compliance. Although the described risks, like hallucinated outputs and excessive use, exist, vibe coding and AIC-powered bots are cheaper and require fewer people to work. The significance of the security practice early on and, associated with it, the real-time threat-adaptive process remains an essential theme of the study, where two most extensively discussed such concepts, federated learning and reinforcement learning are the major facilitators of prospective amelioration.

Talking about everything, the literature underlines what AI has contributed to DevSecOps and the importance of the solutions that could be expanded, reproduced, and explained. To attain strong and flexible security automation, which are the basic principles supporting the objectives of an effective AI-enhanced DevSecOps model, additional studies are essential in terms of moral regulations, performance maximization, and adaptive threat modelling.

Chapter 3

Final Specifications, Requirements, and Impact Assessment

3.1 Final Specifications and Requirements

This chapter determines the completed specifications and operational needs of the proposed explainable anomaly detection model that will be used on the DevSecOps test phase. The system leverages LogBERT to evaluate token-level anomalies in DevSecOps logs and analyze log sequences. It integrates Explainable AI and Retrieval-Augmented Generation (RAG) methods to ensure transparency, reliability, and traceability of automated security decisions.(as shown in Figure 3.1).

The framework uses structured DevSecOps log sequences and can identify anomaly-driving tokens, detect anomalous log patterns, and provide human-readable explanations of these behaviors and corrective recommendations. The specifications in this chapter are the finalized design following system implementation and experimentation.

3.1.1 Functional Requirements

The functional requirements of the system are satisfied as follows:

- Process and analyze DevSecOps test-phase logs of different sources.
- Standardize heterogeneous log formats into structured token sequences.
- Analyze log sequences using the LogBERT transformer model to assign token-level anomaly scores.
- Evaluate each token in the log sequences and assign corresponding anomaly scores.
- Use SHAP for token-level anomaly explanations.
- Use RAG to retrieve semantically similar historical log sequences.

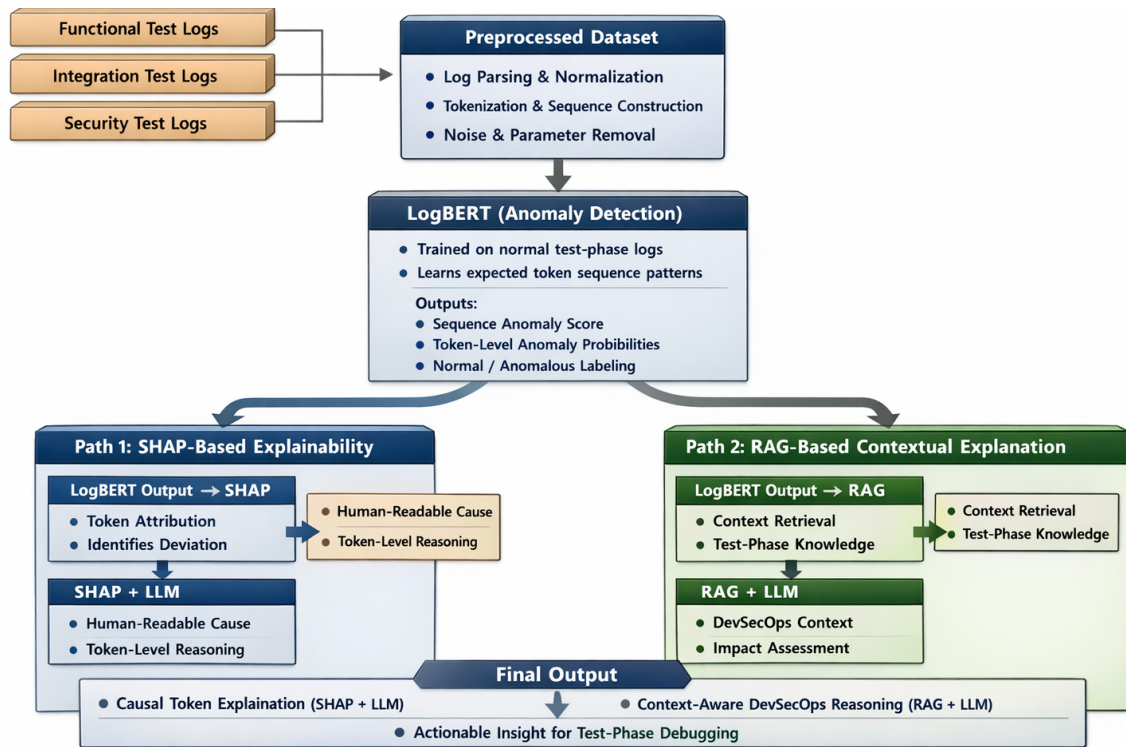


Figure 3.1: DevSecOps Test-Phase Explainability Pipeline

- Create codified descriptions and suggested resolutions with the assistance of an LLM.
- Generate readable reports on anomalies to be used in CI/CD.

3.1.2 Non-Functional Requirements

The non-functional specifications involve:

- Anomaly predictions are highly interpretable.
- Evidence-based transparency.
- Abilitation of results based on stored artifacts.
- Handling large-volume DevSecOps logs.
- Safe management of sensitive system identifiers.
- Long-term updates are maintained in a sustainable codebase.
- Low-latency real-time CI/CD pipeline processing.

3.2 Societal Impact

The offered framework makes a positive contribution to current software engineering practices by enhancing the reliability and trustworthiness of DevSecOps automation. Conventional anomaly behaviour detection systems tend to raise warnings without justification, resulting in slow investigation and low confidence among engineers.

The system does not only enhance human knowledge on anomalous patterns of logs by providing explainable interpretation of the pattern, which is based on historical logs, but also offers tokenlevel evidence. This is useful in making decisions faster, minimises manual debugging time, and encourages collaboration between development and security teams.

There are other significant contributions in society such as:

- Earlier detection of anomalies in software services, ensuring better stability.
- Reduced engineering workload on DevSecOps teams.
- More trust in security instruments through AI assistance.
- More accountability: specifying and verifying decisions.

3.3 Safety Impact

Safety, in the DevSecOps test stage, is associated with software system reliability. The failures applied in the process of CI/CD testing can be transferred into the production environments, which can lead to security breaches, system downtime, and the loss of data.

The proposed framework enhances operational safety since abnormal behavior is identified at the test stage before deployment. Token-level explanations assist engineers in avoiding faulty steps, wrong settings, or security violations. Safety benefits include:

Safety benefits include:

- Prevention of unstable or insecure deployments.
- Early identification of faulty arrangements and pipelines.
- Minimization of cascading CI/CD failures.
- Improved confidence in release validation decisions.

3.4 Legal Impact

Software systems in controlled environments should be traceable and accountable for automated decisions. Black-box AI models present legal risks, as anomaly alerts may not be justified or auditable.

The proposed explainable framework minimizes such risks by providing evidence-based reasoning for all identified anomalies. Token-level attribution and historical-log references enable security resources to record the rationale for decisions and ensure alignment with operational policies.

Legal implications covered by the framework include:

- Assessment and compliance reporting.
- Traceability of anomaly decisions.
- Improved historical investigation records.
- Reduced legal risk from automated, non-explainable actions.

Chapter 4

Proposed Methodology

4.1 Design Process or Methodology Overview

This paper detects anomalies of test phase logs in DevSecOps and here we integrate SHAP based interpretation of the logs which will be performed by LOGBERT. The complexity of this method is that it has to operate with heterogeneous and large volumes of data logs generated in the CI/CD pipelines when testing software and security. It is a procedural approach that may be employed in implementing the proposed framework and identifies abnormal deviations in behavioral patterns in logs on DevSecOps, provides token-level and contextual as well as evidence-based and readable insights, and remediation guidance is to be undertaken. Through this, the methodology directly responds to the research objectives and the research questions of the previous section.

Our target is to build a model that can detect anomalies of logs and explain them accordingly. So, this model can identify anomalous patterns on multi-source DevSecOps logs that provide explicit token-level clarification of model decisions. This methodology will follow a systematic procedure which will include:

1. **Data Collection and Categorization:** Here we collected different publicly available data on the activities of DevSecOps testing including functional, integration and security testing.
2. **Preprocessing and Transformation:** Heterogeneous logs are normalized and then tokenized into numeric forms which are used as inputs of the LOGBERT.
3. **Example Training and Evaluation:** LOGBERT training of identifying token level anomalies and model performance of accuracy, precision, recall and F1-score.
4. **Explainability and Visualization:** SHAP has been implemented to explain the outputs of the trained model which is used to optimize interpretation of the final model output token wise.

The system provides a systematic and explainable process of identifying anomalies during DevSecOps testing phase and it supports explainable security reasoning and CI/CD failure analysis.

4.1.1 Problem Definition and Objective Formulation

During the DevSecOps testing phase, logs are produced which contain critical context on system behavior, vulnerabilities, and operational breakdowns after unit tests, integration tests, functional testing, security scans, deploying and monitoring. Analyzing these logs is important to ensure the reliability, security and compliance of the pipelines.

The problem can be formulated as a token-level anomaly detection problem in which the tokens in a log sequence are classified as 'normal' or 'anomaly'. The aim is to implement a heterogeneous log type pipeline, train LogBERT to learn sequence and token dependency between heterogeneous logs and make the prediction interpretable with SHAP in order to clarify the anomaly prediction with respect to DevSecOps testing phases.

To make this concrete, consider the following example:

Normal log sequence:

[<CLS>, build, completed, successfully, for, moduleA, <SEP>]

Anomalous log sequence (security-related):

["<CLS>", "build", "failed", "due", "to", "unauthorized", "access", "<SEP>"]

Each of these are flagged as out-of-distribution , which means there is a suspected security breach which occurred during the build process using the token "unauthorized." Also, anomalies that are functional or test-related, such as failed test steps, exceptions or errors on the pipeline may be present.

This study will have the following objectives:

- To create a preprocessing pipeline that is able to cope with logs of different types
- LogBert is trained to acquire token level dependencies between various types of logs.
- At this point, interpretable outputs of SHAP might be visualized as tokens of influence that cause the anomaly predictions.

Such an approach can provide the ability not only to identify anomalies correctly but even down to the underlying causes that can be taken action on and leveraged to inform the decision-making in DevSecOps pipelines.

4.1.2 Sub-Phases of DevSecOps Test Phase

DevSecOps test assures functionality, compliance, quality, and security of code, applications and infrastructure and verifies them before deployment. Continuous testing is part of CI/CD pipelines, which allows detecting defects, vulnerabilities and performance in time. The given methodology is aimed at analyzing and explaining actions in this stage based on the processing of the logs of various testing activities.

The sub-phases that are described in the methodology include:

- Unit Testing and Vulnerability Scanning: Scan and check against the vulnerabilities.
- Integration Testing: Inter-service and inter-module testing.
- Functional Testing and Bug Finding: Confirm the characteristics of the application and find bugs.
- Security Testing (Static, Attack, and Threat Detection): Examine the code and test for possible attacks.
- Vulnerable Code Patterns and Dangerous Libraries: Static Analysis and Dependency Scanning.
- CI/CD Build and Test Failures: Log pipeline failures and runs.
- Intrusion Detection (IDS Logs): Monitor network traffic and system logs.
- Firewall Security Testing: Apply and audit network policies.
- Application Layer Security and Behavior Monitoring: Track processes to detect anomalies.
- Live Test Logs: Accept traces to check health and errors.
- Monitoring: Look for abnormal or misuse of resources.

These sub-phases offer an overall coverage of quality of code, security, reliability of operations, and performance of the system which can render the detection and mitigation of defects and security threats proactively.

4.2 Data Collection

A total of seven (7) publicly accessible datasets, with more than **40,000 sequences**, have been gathered and clustered into three categories of data relevant to **DevSecOps**:

- **CyberGuard Network Logs and IDS**

- **Type:** Security Surveillance and Threat Identification.
- **Description:** specializes in intrusion detection, firewall and system monitoring through providing structured logs with which to interact with threats and guarantee interpretable logic. The data is also aimed at construction of IDS, firewall, and application-level security analysis, and understandable high interpretability anomaly detection models.

- **CodeSecure QA Logs**

- **Type:** Testing and CI/CD Pipeline Analysis.
- **Description:** Includes unit, integration, functional, and security testing logs, along with CI/CD build and test results. These datasets offer formal test outcomes and pipeline wellbeing measurements in order to help determine failures and weaknesses. DevSecOps pipelines retrieval based query and traceability are especially appropriate with the logs.

- **TravisSecure Logs**

- **Type:** Comprehensive Build, Deploy, and Security Logging.
- **Description:** Covers build and deployment processes, unit testing, integration testing, and security testing, intrusion testing, firewall activity and system monitoring. The data sets comprise precise logs during the whole lifecycle of the DevSecOps, which can be employed in the analysis of the testing and security fields through AI.

data collection strategy is restricted to using these raw data sets collected publicly available repositories (e.g., **GitHub**) to provide variety in the type of logs and be able to address all the steps of the **DevSecOps** process and be consistent with the real-life CI/CD and security setup.

4.2.1 Data Cleaning

The purpose of the preprocessing pipeline was to convert the heterogeneous **DevSecOps** test-phase logs into the structured, interpretable and BERT-ready to work with **LogBERT** datasets. The workflow at the same time guaranteed maintenance of both numeric representations (which were required as model input) and mapped token sequences (which were required as interpretability and explainability).

1. **Download Raw Data:**

Raw logs data were retrieved in the public repositories either in compressed or plain-text file (.zip, .log, .txt). The files were unbundled and arranged into directories in order to be processed.

2. **Load Raw Logs:**

The log files were sequentially read and each line was considered to be a particular event where there was a timestamps, level of log, message, and parameters. This ensured the close maintenance of tokens to be studied in the future.

3. Tokenize Logs:

Raw log messages were broken up into tokens, including the syntactic and semantic structure of the message. The tokenization was used to isolate categorical keywords (e.g., *deploy*, *error*, *passed*) and runtime-dependent parameters (ID, file path, IP address, etc).

Token classes introduced:

- **Structural Tokens:** `<CLS>`, `<SEP>`, `<PAD>`, `<UNK>` for sequence management.
- **Categorical Tokens:** Pipeline activities (*build*, *scan*, *deploy*), system states (*success*, *failed*), and security events (*attack*, *threat*).
- **Specialized Domain Tokens:** *protocol_type*, *browser_type*, *attack_signature*, consistently mapped from log attributes.

Example:

```
Raw log: "Deploy completed for build 94836 at 2025-03-11T14:07Z"
Tokenized: ["<CLS>", "deploy", "completed", "for", "build",
"<param>", "at", "<param>", "<SEP>"]
```

4. Normalize Variable Parts:

Quantitative attributes (e.g., *cpu_usage*, *memory_usage*, *session_duration*, *packet_size*) were normalized to the $[0, 1]$ range using **min-max normalization**:

$$x_{\text{norm}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

Out-of-bound or missing values were clipped or substituted with `<param>` to ensure no loss of sequence continuity.

4.2.2 Data Transformation

5. Map Tokens to Numeric IDs:

To get log sequences into a format that is usable to feed a model, a vocabulary mapping was made that assigned every token a unique numeric representation, but maintained the original token associations to allow interpretation.

For example, special tokens such as `<CLS>`, `<SEP>`, `<PAD>`, `<UNK>`, and placeholder tokens like `<param>` were assigned IDs **0**, **1**, **2**, **3**, and **4** respectively. Meaningful log keywords such as *deploy*, *completed*, *error*, and *pass* were mapped to IDs **42**, **56**, **78**, and **81**.

Example:

```
Raw log: "Deploy completed for build 94836 at 2025-03-11T14:07Z"
Tokenized: ["<CLS>", "deploy", "completed", "for", "build",
"<param>", "at", "<param>", "<SEP>"]
Numeric: [0, 42, 56, 5, 87, 4, 22, 4, 1]
```

This dual representation strategy allows **LogBERT** to efficiently learn patterns from compact numeric vectors while maintaining traceability to the original tokens, which is essential for post-hoc explainability using **SHAP**, enabling token-level attribution of anomalies in **DevSecOps** logs.

6. Sequence Standardization:

To accommodate all log sequences to a fixed maximum sequence length in **LogBERT**, a structured sequence normalization procedure was applied:

- **Truncation:** Sequences exceeding the maximum permissible length were trimmed, while significant structural tokens such as **<CLS>** (start of sequence) and **<SEP>** (end of sequence) were retained. This ensures that information about sequence boundaries is preserved, which is vital for correctly interpreting token-level dependencies and positional information inside the transformer model.
- **Padding:** Shorter sequences were padded with tokens representing the numerical ID **2** (**<PAD>**). Padding allows efficient batch processing during both training and inference, helping the model handle sequences of different lengths consistently.
- **Attention Masking:** Padding tokens were masked in the attention mechanism to prevent unnecessary computations. This ensures that the model focuses only on meaningful tokens and avoids introducing noise or bias into contextual representations.

Through this process, the semantic and structural integrity of the logs was maintained, enabling **LogBERT** to effectively learn the dynamics of variable-length sequences without the interference of artificial padding.

7. Generate Labels:

Each token in a log sequence was labeled according to its role in detecting anomalies:

normal event — 0 **anomalous event** — 1 **irrelevant token** — -1
(padding, metadata, etc.)

Example:

Token label sequence: [1, 1, -1, 0, 1, 0, -1]

Interpretation: The first two tokens represent anomalies (e.g., deployment failure, security alert), the third is ignored padding, the fourth is a normal event, the fifth is another anomaly, the sixth is normal, and the seventh is ignored padding.

This design enables **LogBERT** to recognize relevant events and maintain traceability for post-hoc interpretability using **SHAP**.

Ground Truth Label Generation:

Ground truth labels for token-level anomaly detection were generated using a multi-faceted approach to ensure accuracy, coverage, and model generalization:

- **Manual Annotation:** Security and **DevSecOps** experts reviewed logs to identify anomalies such as intrusion attempts, failed test cases, or abnormal system behaviors, providing high-quality human-labeled data.
- **Automated Heuristics:** Frequent log patterns indicating abnormal events (e.g., *unauthorized access*, *error*, *timeout*) were programmatically flagged as anomalous, enabling rapid labeling of large datasets while maintaining consistency.
- **Synthetic Injection:** To balance class distributions and expose **LogBERT** to rare or extreme anomalies, controlled anomalous tokens were inserted into otherwise normal sequences. This helped the model learn to detect infrequent but critical events.

This combined strategy ensured that each token in a log sequence was accurately labeled as normal, anomalous, or irrelevant, enabling **LogBERT** to perform precise token-level anomaly detection while preserving traceability for explainability through **SHAP** and **RAG**.

4.2.3 Data Reduction

In order to prevent vocabulary explosion and enhance computational efficiency on GPU, tokens that have high entropy like session id, unique hash, dynamic timestamps and other highly varying tokens were selectively substituted with a special token `!param?`. This is used to retain the overall structure and context of log sequence sequences but significantly decreases the size of distinct tokens that LogBERT needs to be concerned with. This removes such noisy or uninformative variability hence the model is able to focus on meaningful semantic patterns, operational behaviours and anomalous events, which can offer considerable enhancement in the performance of learning and interpretability during token-level anomaly detection.

4.2.4 Final Preprocessed Dataset

The final dataset contained approximately **40,000 structured sequences** collected from CI/CD logs, security testing logs, and system monitoring traces. Each sequence was stored as a structured **JSON object** containing the following components:

- **sequence_id:** A unique identifier assigned to each log sequence for tracking and reference.
- **numeric_sequence:** The sequence of numeric IDs corresponding to each token, used as direct input to the **LogBERT** model.
- **mapped_sequence:** The original token sequence preserved for interpretability, enabling post-hoc analysis and **SHAP**-based explanations.
- **label:** Ground truth anomaly labels for each token, indicating whether it is normal (0), anomalous (1), or ignored (-1).

- **error_anomaly:** A summary flag indicating whether the sequence contains any anomalous tokens, facilitating quick identification of sequences requiring further investigation.

The integration of `<param>` tokens, numeric ID mapping, and `labelmap.json` ensured a **lossless mapping** between symbolic log events and their numerical inputs. All sequences were saved line by line into a **JSON file**, creating a model-ready and explainable dataset.

This complete traceability allows **SHAP** and attention-based visualizations to directly highlight anomalous tokens in their human-readable form, thereby supporting **actionable insights** for **DevSecOps** pipelines.

4.3 Model Architecture and Training

The proposed model learns the structured **DevSecOps** log sequences and predicts anomalous behavior with the help of the LogBERT that is a transformer-based deep learning model, which is based on the BERT architecture. Unlike the rule-based approach of the log analysis, **LogBERT** is trained to learn normal execution patterns based on the activities of discovering the sequential dependencies and contextual relationships between the log events. It has a powerful architecture that allows the modelling of complex patterns of heterogeneous sources of logs, and it generates token-level reconstruction opportunities to study anomaly behaviour.

4.3.1 Model Architecture

LogBERT represents each log event as a **tokenized log key**, which is first mapped into a dense vector space via a **trainable embedding layer**. These embeddings preserve semantic and positional information of log events within a sequence. The embedded sequences are then processed through multiple stacked **Transformer encoder layers**, each comprising:

- **Multi-head self-attention:** captures long-range dependencies between log events.
- **Position-wise feedforward networks:** transform contextual embeddings into richer representations.
- **Residual connections and layer normalization:** ensure stable training and better gradient flow.
- **Dropout regularization:** mitigates overfitting and improves generalization across diverse log sources.

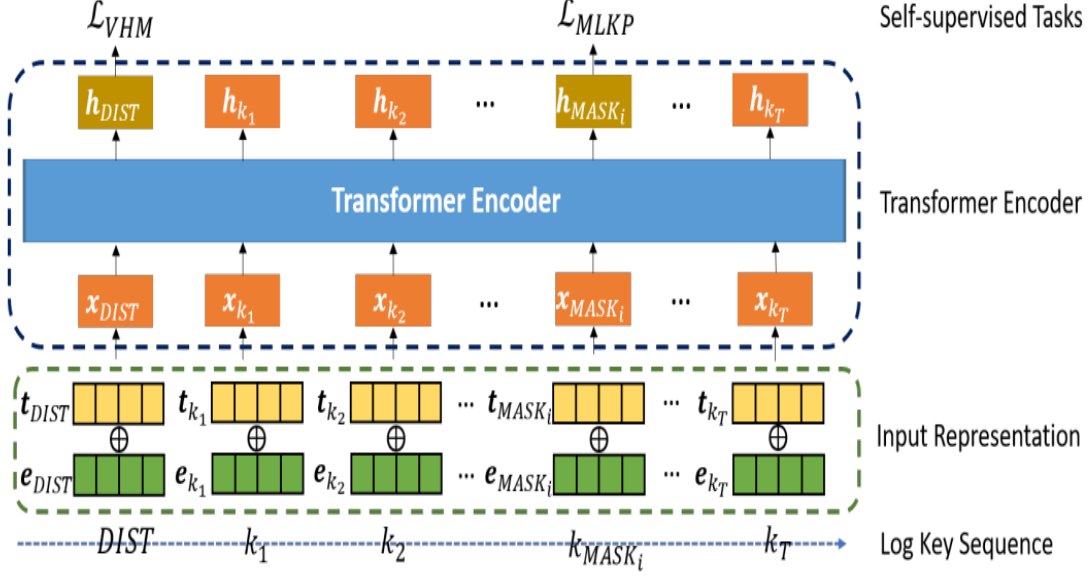


Figure 4.1: LogBERT architecture

As shown in Figure 4.1, the proposed LogBERT architecture processes structured log key sequences through embedding layers and stacked Transformer encoders to capture both sequential and contextual execution patterns.

Through this architecture, LogBERT effectively captures **both local and global execution patterns** in DevSecOps pipeline logs, enabling robust token-level analysis and providing the foundation for explainable anomaly assessment.

4.3.2 Training Strategy

During training, **LogBERT** is trained using a style of **masked language modeling task**, in which the model is trained to approximate the probability of each log token in context. The deviations in the learned normal behavior produce tokens with a greater **negative log-likelihood (NLL)** that measures the degree of deviation at the token level. The higher the NLL the greater the deviation than normal behavior. Higher values of the negative log-likelihood (NLL) are used downstream to detect anomalies, and explainable analyze them.

The training objective is optimized with **cross-entropy loss**, ignoring padding tokens to prevent filler entries from affecting learning. Preprocessed numeric log sequences are used as input, and random seeds are fixed to ensure reproducibility of experiments.

An **80–20 train–test split** is applied, with the test set held completely unseen during training to avoid data leakage. Multiple training runs are performed with varying batch sizes, sequence lengths, hidden dimensions, and epoch counts to determine optimal hyperparameters. The model is optimized using the **Adam optimizer**, and dropout is applied to improve generalization.

Crucially, LogBERT does not predict normal or anomalous labels. Instead, it produces token-level NLL values, which are saved for each log sequence and later used for **SHAP analysis**, **RAG retrieval**, **Human-readable explanation generation**.

All model outputs, including trained weights, token embeddings, and per-token NLL scores, are preserved to ensure downstream explainability, traceability, and reproducibility in DevSecOps test-phase analysis.

1. **model_final.pt — Trained LogBERT Weights:**

This file contains the final PyTorch checkpoint of **LogBERT** after training under the best-performing hyperparameters. The weights capture the learned contextual representations of log sequences and can be directly reused for:

- **Inference:** Analysis of anomalous behavior in unseen test-phase logs without retraining.
- **Explainability:** The foundation for both **SHAP** attribution analysis and **RAG**-based contextual grounding, ensuring that model explanations can be traced back to token-level contributions and historically relevant log sequences.
- **Reproducibility:** Enables the reproduction of results in subsequent experiments or comparisons with other models.

The file also stores the optimizer states and training metadata, making it suitable for fine-tuning or incremental learning in long-running scenarios.

2. **tokenized_inputs.npz — Preprocessed Numeric Sequences:**

The tokenized logs used for training and evaluation are stored as compressed NumPy arrays containing numeric tensor representations of the tokenized logs. Sequences are encoded as integer IDs, truncated or padded to a fixed maximum length based on the label map JSON. The main advantages include:

- **Reproducibility:** Ensures the model is trained and evaluated on the same input data, eliminating randomness.
- **Interpretability:** Supports provide explainability through mapping tokens to their native semantic representations (SHAP) and retrieving useful historical sequence of logs to provide the necessary contextualization (RAG).
- **Efficiency:** Enables reloading large datasets quickly without re-running the preprocessing pipeline, which is ideal for large-scale log analysis.

3. **labelmap.json — Token-to-ID Vocabulary Mapping:**

This JSON file defines the vocabulary space, where every token observed during preprocessing is assigned a unique integer ID. It includes special tokens (<CLS>, <SEP>, <PAD>, <param>) and categorical tokens (e.g., **deploy**, **error**, **scan**). Its significance lies in:

- **Encoding Consistency:** Ensures uniform token-to-ID mapping across training, validation, and test datasets.

- **Explainability:** The numeric outputs decoded can be rebuilt into understandable token sequences to be displayed on **SHAP** and interpreted into context with **RAG**.
 - **Cross-Experiment Reproducibility:** Prevents vocabulary drift across extended experiments or when integrating future datasets.
4. **config.json — Model Architecture and Hyperparameters:**
 This configuration file documents the complete architecture and hyperparameters of the best-performing experiment, including sequence length, batch size, number of transformer layers, number of attention heads, hidden dimensions, epochs, learning rate, and dropout rate. Its purpose includes:
- **Transparency:** Provides full documentation of the experiment design.
 - **Reproducibility:** Enables retraining under identical conditions to achieve the same results.
 - **Comparability:** Serves as a baseline for ablation studies or comparisons with alternative architectures.
5. **tokenizer/ — Tokenization Utilities:**
 This directory contains scripts and tools for translating raw log text into corresponding numeric IDs. It includes vocabulary files, encoding/decoding functions, and preprocessing scripts. Its importance lies in:
- **Interpretability:** Provides that **SHAP** descriptions and **RAG**-based contextual knowledge be readable by humans.
 - **Pipeline Continuity:** Guarantees consistent preprocessing for new incoming logs, matching the original training data.
 - **Future Extensions:** Facilitates vocabulary expansion to accommodate new log types or tokens introduced in evolving **DevSecOps** workflows.

This training procedure was used to get optimal explainability that mitigates the doubts developer have during **DevSecOps test phase**. It is based on the trained LogBERT model that the **Explainable AI (XAI)** modules are built in, which allows reconstruction and visualization of log sequences, token-level attribution with **SHAP**, contextual grounding with **RAG**, and produces human-readable explanations that help increase interpretability, traceability, and understanding of pipeline behavior.

4.3.3 Evaluation

The trained LogBERT model was evaluated on three datasets covering different DevSecOps Test Phase sub-phases, including security monitoring, unit and integration testing, CI/CD build/test monitoring, and system performance anomalies.

The table summarizes performance metrics (accuracy, F1-score, precision, and recall) across the datasets, while the per-epoch diagram shows stable learning dynamics with steadily increasing validation accuracy and decreasing validation loss, indicating effective model convergence without overfitting.

Dataset	Category	Accuracy (%)	F1 (%)	Precision (%)	Recall (%)
CyberGuard IDS and Network Logs	Security Monitoring, Threat Detection	93.750	91.667	90.625	93.750
CodeSecure QA Logs	Testing, CI/CD Pipeline Analysis	91.018	82.925	85.146	85.515
TravisSecure Logs	Comprehensive Build, Deploy, Benchmarking	90.443	73.258	72.982	74.470

Table 4.1: Token-wise LogBERT Performance Evaluation

Interpretation:

- **CyberGuard IDS and Network Logs:** LogBERT achieved the highest performance on structured security logs, accurately detecting intrusion patterns and abnormal events, highlighting the model’s ability to handle well-defined attack sequences.
- **CodeSecure QA Logs:** The model was robust in identifying functional defects, failed tests, and pipeline anomalies despite differences in the testing and pipeline logs, indicating that the model could identify anomalies in heterogeneous DevSecOps test data.
- **TravisSecure Logs:** LogBERT obtained good anomaly results on a series of build, deployment, and monitoring logs, at a slightly lower precision and recall due to the high log variability which is closer to the real-world scenario in multi-domain logs.

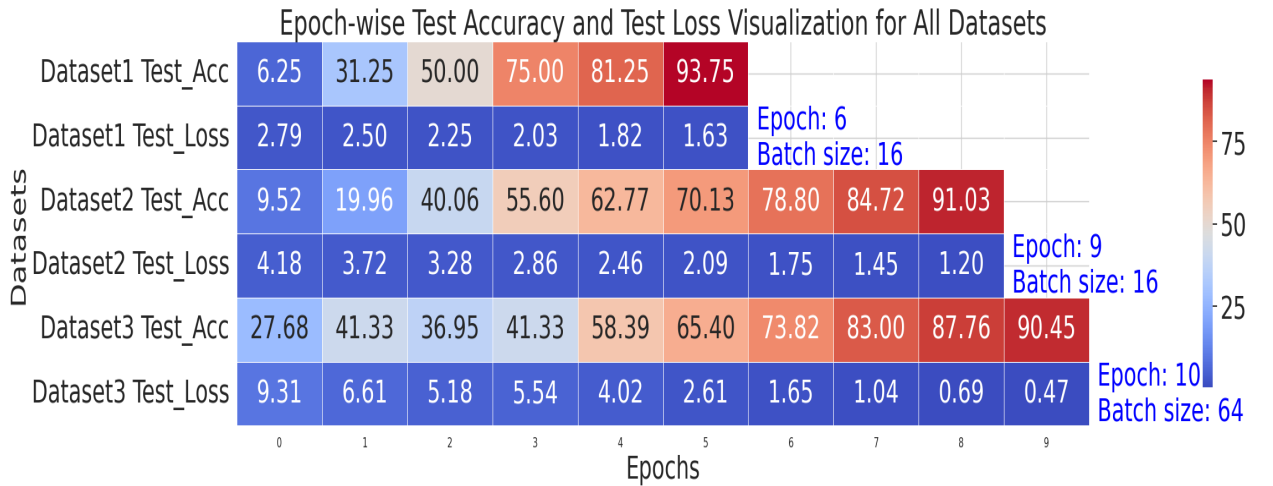


Figure 4.2: LogBERT Training Performance by Epoch

The Table 4.1 provides the analysis of three datasets with the final accuracy, F1 score, and recall. These metrics give a summary of the performance of the LogBERT which was trained on these datasets. Besides, Figure 4.2 demonstrates the accuracy of the test, and the losses per epoch of the three datasets, and the batch size at which training occurs. Based on these outcomes, it is possible to note that the model is neither overfitting nor underfitting, and the learning process is proceeding as intended.

In general, LogBERT offers consistent token-level anomaly detection in several DevSecOps test-phase sub-phases. The trends and measures of the per-epoch training approach prove the compatibility of the model in learning sequence patterns and contextual dependencies, which offer a strong base of SHAP-based explainability and actionable findings in the context of DevSecOps work.

4.3.4 LogBERT-Based Anomaly Detection Results

The proposed framework of anomaly detection based on LogBERT was tested on a per-log basis with a test set that contained more anomalous logs (10,837 anomalies and 1,425 normal logs). This evaluation configuration was selected intentionally, with anomaly detection in DevSecOps test-phase pipelines focusing on the consistent determination of abnormal execution behavior over the balanced classification accuracy.

Confusion Matrix Analysis

The initial confusion matrix plot (shown in Figure 4.4) is the combination of the absolute counts and a row-percentage percentages, which gives a good representation of the model performance in relation to each class. From the figure:

- **True Positives (TP):** 8,658 anomalies correctly detected ($\approx 79.9\%$ of actual anomalies)
- **False Negatives (FN):** 2,179 anomalies not detected ($\approx 20.1\%$ of actual anomalies)
- **True Negatives (TN):** 275 normal logs correctly classified ($\approx 19.3\%$ of actual normals)
- **False Positives (FP):** 1,150 normal logs misclassified as anomalies ($\approx 80.7\%$ of actual normals)

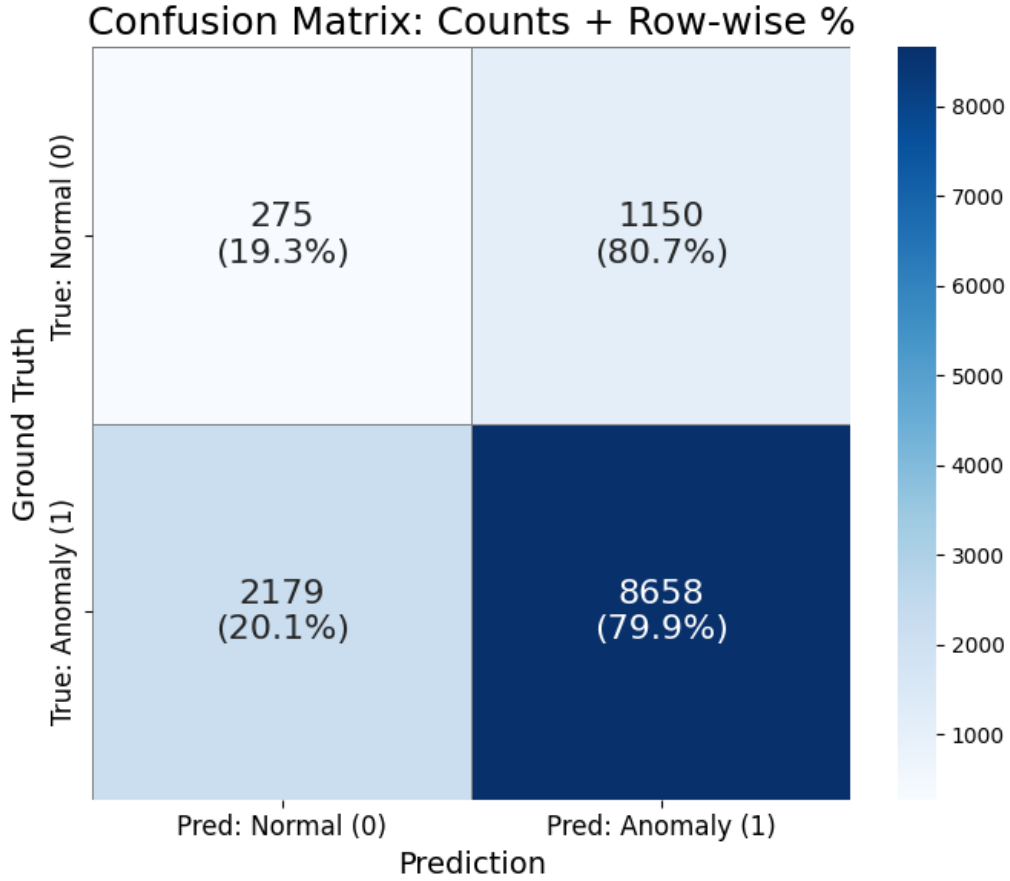


Figure 4.3: Confusion matrix showing raw prediction counts

Evaluation Metrics

The results of the confusion matrix evaluation metrics are introduced in Figure 4.4. These findings can be used to show that there are high levels of performance in terms of precision and equal recall in the case of anomaly detection.

The anomaly decision threshold was determined by the best F1-score, which provided a good compromise between false positives and false negatives in the case of imbalanced data. High precision means that the majority of anomalies detected represent real abnormal behavior, and an individual will not be alerted unnecessarily when performing test-phase analysis. The obtained recall proves that a large percentage of the deviant executions is effectively detected.

The missed anomalies are predisposed to a number of factors. There are also anomalous sequences whose pattern of execution is typical of normal behavior, leading to low reconstruction error. Also, anomalies whose scores are near the optimized threshold can fall under the decision boundary. Due to the fact that LogBERT works in an unsupervised learning context, the probability of rare but structurally valid abnormal patterns being given high likelihood estimates remains, decreasing their score on the anomaly distribution.

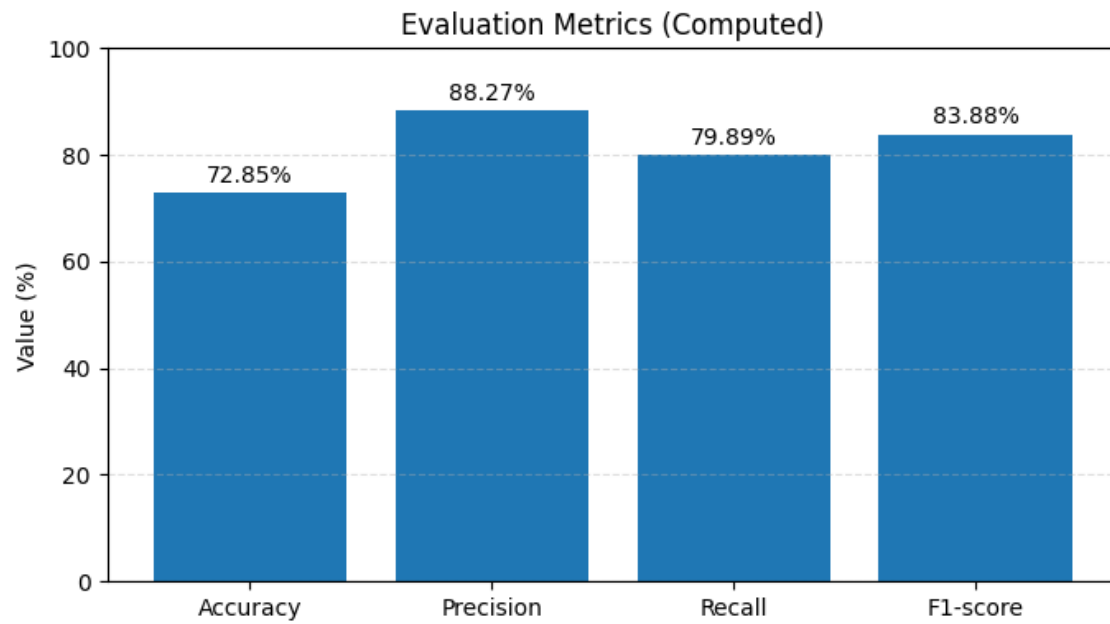


Figure 4.4: Evaluation Metrics

The intentionally biased dataset impacts the overall accuracy and cannot be viewed as the major performance measure. Rather, the trade off between precision and recall gives a more valuable measurement of the detection reliability.

This is because the precision (88.27) is high, which implies that the majority of the identified anomalies are true.

Metric	Interpretation
Precision	Most detected anomalies are true anomalies
Recall	Large percentage of real anomalies are identified
F1-Score	Balance between precision and recall is high
Overall Accuracy	General correct predictions across all logs

Table 4.2: Interpretation of LogBERT Evaluation Metrics

The experimental findings support that LogBERT is able to detect anomalous execution behavior in test-phase logs at DevSecOps. The confusion matrices and the evaluation metrics show that there is reliable detection ability in the case of class imbalance which confirms that LogBERT is suitable in anomaly detection with test-phase pipeline monitoring conditions.

4.4 SHAP- and LLM-Based Explainability

In the test phase of the DevSecOps, LogBERT allocates token-level anomaly scores. Each token contributes a value which is measured by SHAP(XAI) and the Mistral-7B-Instruct LLM transforms these values into brief human-understandable explanations with cause and effect logic and proposed remedial actions to allow root-cause analysis.

4.4.1 SHAP-Based Explainability

During the test phase of the DevSecOps, SHAP is used to the masked language modeling (MLM) outputs of the LogBERT to understand the contribution of each character or token to the anomaly. LogBERT gives per-token negative log-likelihood (NLL) scores, and SHAP is permutation-based in quantifying the contribution of individual tokens to the overall anomaly. The result is signed SHAP values and normalized percentages, which points out the most unusual tokens, which are bundled into word-sized spans and recombined into human-readable log previews to gain insight into the tokens, token by token.

Input Processing and SHAP Computation

Input Processing: The unfinished log sequences are then transformed to character-level tokens in order to keep low-level execution information that is common in DevSecOps pipelines. Each sequence is then padded or truncated to a common size of 128 tokens, which has been demonstrated to be able to be logged to a fixed-size input of logBERT, as well as make test-phase logs comparable to one another.

Permutation-Based SHAP Analysis: The permutation-based SHAP explainer is used to explain anomaly predictions using a representative background set of normal pipeline logs to explain the anomaly predictions. The explainer sequentially removes tokens and monitors the subsequent change in the token-wise anomaly scores of LogBERT which permits the quantification of the contribution of each individual token to anomalous behavior in detail.

Token-Level Attribution and Aggregation: SHAP produces signed values of attribution, which are used to determine whether the token increases or reduces the likelihood of anomaly. These values have been normalized into contributions that are in the form of percentages. Very powerful neighboring tokens are then joined into word-size spans, making it possible to identify meaningful anomaly-driving elements like failed tests, vulnerable packages or misconfigured commands.

Human-Readable Reconstruction: Lastly, the log sequences are restored to form readable previews with the most influential tokens and compiled spans enhanced. The structured output has a token-level, interpretability at the detail and can be read by the developers and security engineers in DevSecOps testing, at the test phase, to detect anomalies and their underlying causes.

Sequence Reconstruction and Output

The sequences are reassembled into a human readable preview of the logs and the tokens and the SHAP percentages are stored in an organized format. In each sequence record it has:

- Token-level contributions (signed SHAP values)
- Percentage contribution of each token
- Top word-like anomaly-driving spans
- Reconstructed log preview

Token	—SHAP— (abs)	Anomaly Contribution (%)
package	0.0647914052	24.31 %
fail	0.0463587195	17.39 %
fail	0.0340117544	12.76 %
UNK	0.0293433815	11.01 %
0	0.0179915637	6.75 %
32	0.0175263941	6.58 %
229	0.0169312447	6.35 %
UNK	0.0132662490	4.98 %
deploy	0.0131591141	4.94 %
2169	0.0131262571	4.92 %
TOTAL	0.2665060833	100 %

Table 4.3: Sequence 12 — Word-Level Token Anomaly Contribution (SHAP %)

This allows the exact token-wise interpretability of the anomaly detection output of LogBERT, such as what characters or tokens are contributing to the anomaly score of each sequence (Shown in Table 4.3).

Interpretation and Visualization

In the SHAP-based interpretability visualization, color intensity reflects how each token contributes to the model’s anomaly prediction. Here, blue indicates a SHAP contribution toward normal behavior, while red indicates the intensity of anomaly influence. Figure 4.6 and Figure 4.5 show token-level SHAP visualizations across multiple and single log sequences, respectively.

Blue regions – SHAP Contribution (Normal Behavior): Blue tones represent negative or baseline SHAP values, meaning the token contributes to normal or expected system behavior, thereby reducing the anomaly probability.

- **Dark blue:** Shows negligible SHAP influence, typically associated with neutral or structural tokens such as `¡PAD¿`, `¡param¿`, or routine log entries like "process started", which do not affect anomaly detection.
- **Light blue:** Indicates a slight negative SHAP contribution, reinforcing normal operation and helping the model avoid false positives. For example, the token "login successful" may appear in light blue, signifying that this event is common and supports normal system behavior.

Red regions – Anomaly Intensity (Abnormal Behavior): Red tones correspond to positive SHAP values, meaning the token increases the anomaly intensity and raises the likelihood of being classified as abnormal.

- **Light red:** Represents a minor anomaly impact, typically caused by infrequent or slightly irregular log patterns, such as a normal "cache refresh initiated" event that deviates slightly from expected patterns.
- **Dark red:** Indicates strong anomalous influence—tokens that directly correspond to critical or security-sensitive events. For instance, the token "unauthorized access" may appear in dark red, indicating a high SHAP value and a strong contribution to anomaly prediction by the model.

In summary, blue areas represent SHAP contributions toward normal behavior, while red areas indicate the severity of anomalies and highlight tokens that drive the model toward abnormal predictions. For example, "login successful" (light blue) represents system stability, whereas "unauthorized access" (dark red) demonstrates a strong positive SHAP contribution, signaling a significant anomaly.

4.4.2 LLM-Enhanced SHAP Explainability (SHAP+LLM)

Whereas SHAP finds the contribution of tokens, dense SHAP results are difficult to interpret manually. In order to enhance usability and working efficiency, the Mistral-7B-Instruct Large Language Model is used to produce structured, human-readable explanations of the SHAP results.

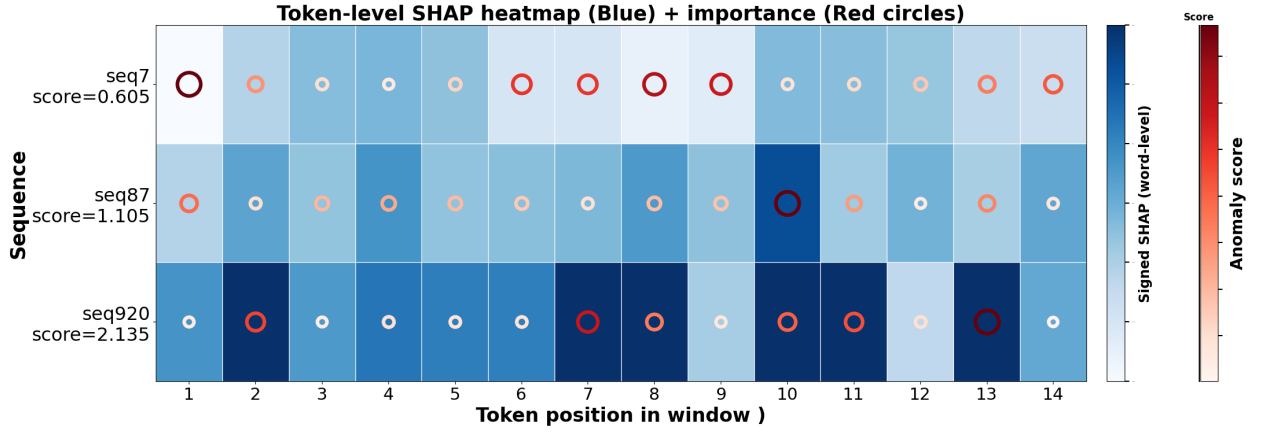


Figure 4.5: Token-level SHAP visualization across three log sequences.

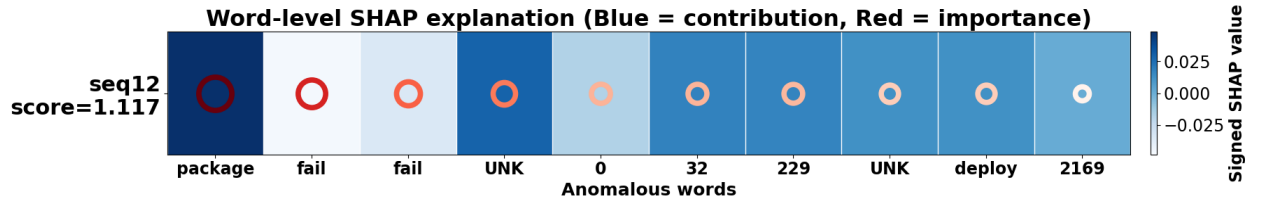


Figure 4.6: Token-level SHAP explanation for a single log sequence.

LLM Selection and Justification

Mistral-7B-Instruct is chosen because it is both an effective instruction-following and an effective low-precision deployment model, and as such it can produce dense SHAP token-level outputs that can be interpreted in real-time, and no hallucination. Reasons behind this model can be applied: Unlike other LLMs, Mistral-7B-Instruct will always transform token-based anomaly scores of LogBERT into brief and structured explanations during the DevSecOps testing phase. It preserves the fine-grained SHAP attribution, highlights the essential anomalous tokens, and provides actionable information to the developers and security engineers, which accelerates the analysis of the anomalies as well as their more precise and repeatable in the CI/CD pipeline.

LLM Workflow(with SHAP)

Top-K anomaly-driving tokens per log sequence are directly extracted based on the outputs of SHAP, which points out the tokens or characters which have the strongest effect on the anomaly score. All these tokens are then linked to concrete log records where they occur, creating an "Issue" section which summarizes the main drivers of anomalous behavior.

The structured prompt provided to the LLM requires the LLM to give two outputs, one a **Reason**, in which it is requested to provide short and token-specific causal explanations, and a **Suggested Fix**, in which it is requested to provide a few actionable remediation steps. The Mistral-7B-Instruct model operates on the token-level SHAP representation and converts it to the short human-understandable explanations sacrificing interpretability, but gaining clarity.

Lastly, the result is structured by integrating sequence metadata, the issue block and reasoning and remediation steps generated by the LLM into one and only actionable report and is therefore appropriate to be fully built into DevSecOps test-phase pipelines. The SHAP and LLM explainability pipeline for DevSecOps test-phase logs is illustrated in Figure 4.7

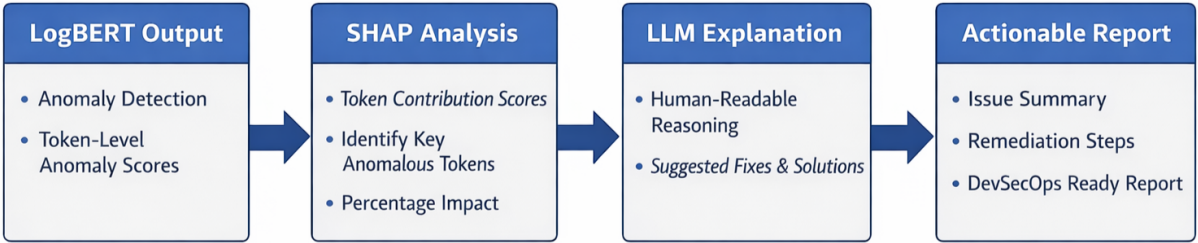


Figure 4.7: SHAP + LLM explainability pipeline

SHAP+LLM Example

Example-1

Sequence ID: 914 — Anomalous

Issue: The log indicates the installation of Node.js 18 and running npm packages, ESLint, unit tests, and a dependency scan. An error occurred during the dependency scan, indicating a license violation. The tokens 'dependency', 'packages', 'License', 'violation', 'UNK', 'Install', and 'Run' all appear in the log.

Reason (top anomaly-driving tokens with log reference):

- **dependency** → appears in log as ... Run ESLint<UNK>Run unit tests<UNK>Run dependency scan<UNK>Error: License violation detected
....
- **packages** → appears in log as Set up Node.js 18<UNK>Install npm packages<UNK>Run ESLint<UNK>Run unit tests<UNK>....
- **License** → appears in log as ... ts<UNK>Run dependency scan<UNK>Error: License violation detected<UNK>Resolve c
- **violation** → appears in log as ... un dependency scan<UNK>Error: License violation detected<UNK>Resolve c
- **UNK** → appears in log as Set up Node.js 18<UNK>Install npm packages<UNK>Run ESLint<UNK>....
- **Install** → appears in log as Set up Node.js 18<UNK>Install npm packages<UNK>Run ESLint<UNK>Run u....
- **Run** → appears in log as ... de.js 18<UNK>Install npm packages<UNK>Run ESLint<UNK>Run unit tests<UNK>Run dep ...
and ... stall npm packages<UNK>Run ESLint<UNK>Run unit tests<UNK>Run dependency scan<UNK>....

Suggested fix:

1. Review the installed npm packages for any potential license issues.
2. Update or remove any problematic packages.
3. Run the dependency scan again to ensure all dependencies are properly licensed.

Example-2

Sequence ID: 51 - Anomalous

Issue: The log indicates two failed build processes, each with an associated scan and deploy step. The 'build' token represents the build process, while 'scan' and 'deploy' tokens represent the subsequent steps. The 'fail' token indicates an error or issue during these steps. The 'UNK' token represents unknown values.

Reason (top anomaly-driving tokens with log reference):

- **fail** → appears in log as build 46.86 1367.89 490.8 fail 0 2<UNK>scan 17.75 5532.82 108.03 pass 0 3<UNK>deploy 58.81 1353.48 244.34 fail 1 0<UNK>test 67.75 7458.49 33.
- **build** → appears in log as build 46.86 1367.89 490.8 fail 0 2<UNK>scan
- **UNK** → appears in log as build 46.86 1367.89 490.8 fail 0 2<UNK>scan 17.75 5532.82 108.03 pass 0 3<UNK>....
- **244** → appears in log as03 pass 0 3<UNK>deploy 58.81 1353.48 244.34 fail 1 0<UNK>test 67.75 7458.49 33.
- **pass** → appears in log as ... fail 0 2<UNK>scan 17.75 5532.82 108.03 pass 0 3<UNK>deploy 58.81 1353.48 244.34
- **scan** → appears in log as ... build 46.86 1367.89 490.8 fail 0 2<UNK>scan 17.75 5532.82 108.03 pass 0 3<UNK>deploy
- **deploy** → appears in log as ... scan 17.75 5532.82 108.03 pass 0 3<UNK>deploy 58.81 1353.48 244.34 fail 1 0<UNK>test
- **test** → appears in log as ... test 7458.49 33.

Suggested fix:

1. Investigate the cause of the build errors by reviewing the build logs and error messages.
2. Address any identified issues and retry the build process.
3. Re-run the scan and deploy steps for the successful builds.

Example-3

Sequence ID: 7994 — Anomalous

Issue: The log contains multiple occurrences of the tokens "Repudiandae", "eligendi", "temporibus", and others. These tokens are associated with a DNS query or response, as indicated by the "Data DNS" context. The anomalous behavior is the repetition of these tokens in the log, suggesting a potential DNS query or response loop or a DNS amplification attack.

Reason (top anomaly-driving tokens with log reference):

- **Repudiandae** → appears in log as ... P IP IP 55912 29164 ICMP 776 Data DNS Repudiandae quaerat eligendi neque temporibus. Ni
- **eligendi** → appears in log as ... ICMP 776 Data DNS Repudiandae quaerat eligendi neque temporibus. Nisi culpa fugit. A ... and ... que temporibus. Nisi culpa fugit. Aut eligendi odit
- **TIMESTAMP** → appears in log as TIMESTAMP IP IP 55912 29164 ICMP 776 Data DNS R
- **temporibus** → appears in log as ... NS Repudiandae quaerat eligendi neque temporibus. Nisi culpa fugit. Aut eligendi odit
- **quaerat** → appears in log as ... 2 29164 ICMP 776 Data DNS Repudiandae quaerat eligendi neque temporibus. Nisi culpa
- **fugit** → appears in log as ... eligendi neque temporibus. Nisi culpa fugit. Aut eligendi odit
- **neque** → appears in log as ... Data DNS Repudiandae quaerat eligendi neque temporibus. Nisi culpa fugit. Aut eli
- **odit** → appears in log as ... Data DNS Repudiandae quaerat eligendi neque temporibus. Nisi culpa fugit. Aut eligendi odit

Suggested fix:

1. Implement rate limiting on DNS queries and responses to prevent potential DNS amplification attacks.
2. Configure logging to exclude repetitive DNS query or response logs.
3. Investigate the source of the anomalous DNS traffic and apply appropriate security measures.

4.4.3 Practical DevSecOps Benefits (SHAP + LLM)

By combining SHAP and the Mistral-7B-Instruct model(LLM), the anomaly detection at the DevSecOps test phase is improved since token-level contributions are translated into actionable information. SHAP finds the most important tokens to the anomaly scores of the LogBERT, and the LLM generates structured explanations and remediation options, so the outcomes can be understood and utilized.

Key Benefits:

- **Contextual Anomaly Understanding:** Identifies tokens that impact anomalous behavior so that they can be analyzed in detail regarding their root causes.
- **Human-Readable Explanations:** Converts large SHAP values into concise, structured explanations that are understandable by human analysts and security teams.
- **Actionable Remediation Guidance:** Gives transparent and compact corrective action dependent on identified patterns.
- **Operational Transparency:** Assures interpretability and makes CI/CD pipelines understandable.
- **Pattern and Trend Awareness:** Reveals recurring anomalies so that it can be prevented.

As a SHAP + LLM model, the token-level anomaly detection is interpretable and actionable and enhances efficiency, reliability, and response at the DevSecOps test stage.

4.5 RAG- and LLM-Based Explainability

RAG fetches historical log sequences to give a background context. The manual interpretation of the logs is time-consuming, to make the RAG outputs easier to read and more efficient in their operation, LLM (Mistral-7B-Instruct) is applied to the output. The LLM combines retrieved logs into machine-readable explanations, consisting of causal reasoning and recommended actions, but does not actually carry out anomaly detection. This brings complicated historical context to instant actionability of DevSecOps teams.

4.5.1 RAG-Based Context-Level Explainability

A Retrieval-Augmented Generation (RAG) mechanism is used to explain and it aims at grounding the interpretation of anomalies in historical execution background rather than free-form text generation.

Retrieval Model and Indexing

Semantic representations of log sequences are produced by the SentenceTransformer all-MiniLM-L6-v2 model. The reason behind the choice of this model is that it has done extremely well in short technical text, has low computational costs, and can

be used in real-time CI/CD workloads. The resulting embeddings are indexed with FAISS and allow to find similarities with massive test-phase log repositories.

RAG Explainability Process

The RAG explainability mechanism is a retrieval mechanism that retrieves contextual evidence of the log sequences that are detected as being anomalous by LogBERT. Every sequence is converted into a semantic embedding, and then they are used to identify the most similar historical log sequences. The logs that are retrieved are used as the point of reference, as they connect the present deviation with the observed during the overall executions.

Process Steps(As shown in Figure 4.8):

- **Tokenization:** Divide the anomalous sequence of logs in separated tokens.
- **Embedding Generation:** The contextual and semantic connections in the tokenized sequence are to be encoded by SentenceTransformer all-MiniLM-L6-v2 to encode the sequence.
- **Query Repository:** Search a historical log embedding repository, which was pre-computed using the same model, using the embedding.
- **FAISS Retrieval:** Find top-K nearest neighbor search with the FAISS index by L2 distance.
- **Collect References:** Recall the related historical log sequences as the contextual references of the abnormal sequence.
- **Output Association:** Display the retrieved sequences and the existing anomaly, which gives predictable behavior, repeatable results, embedding-based contextual evidence.

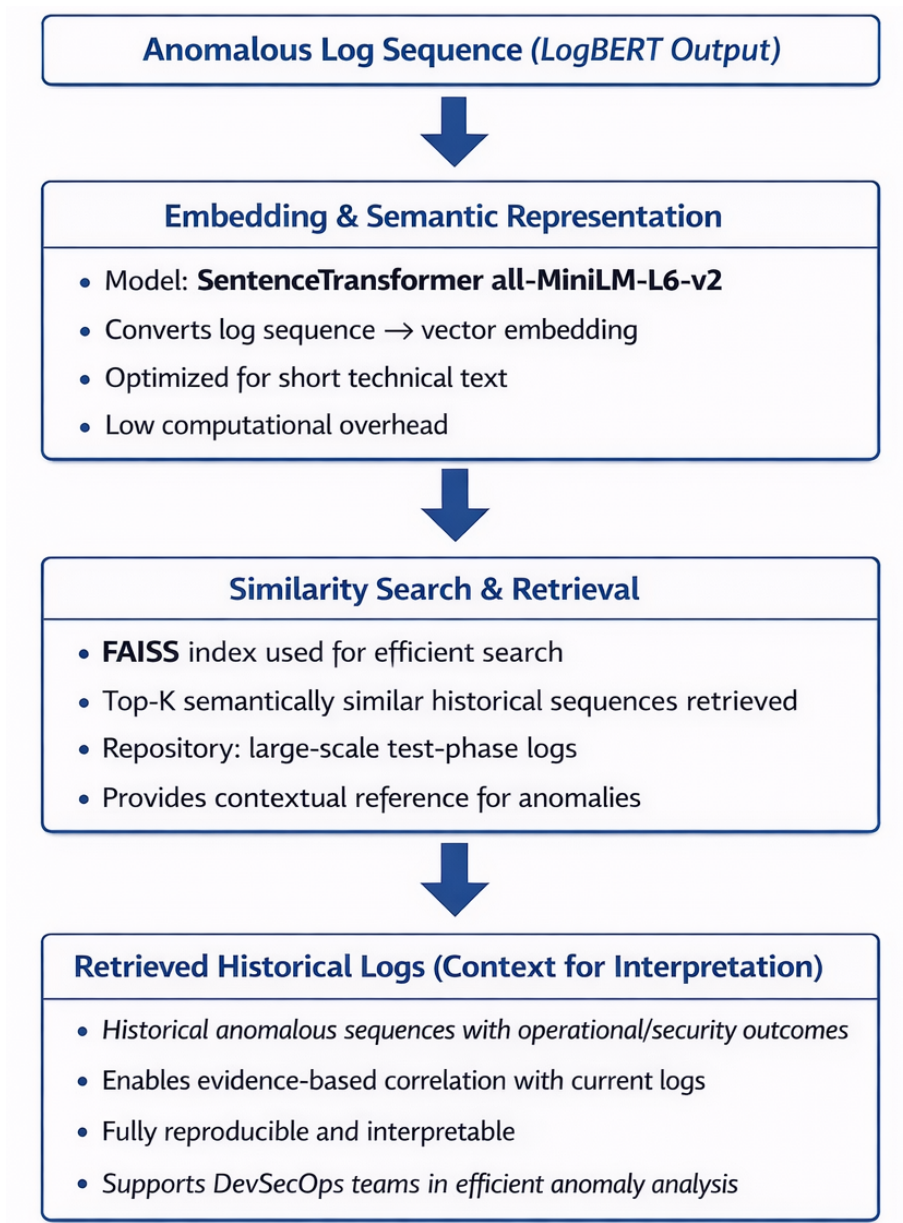


Figure 4.8: RAG Explainability Workflow in DevSecOps Test Phase

RAG Example

Example 1

Sequence ID: 7994
Classification: Anomalous
Historical seq: 6860,30989
Retrieved similar logs:
TIMESTAMP IP IP 30989 7176 ICMP 491 Data FTP
TIMESTAMP IP IP 14013 43376 TCP 301 Control DNS
TIMESTAMP IP IP 22249 5090 TCP 927 Control DNS

Example 2

```
Sequence ID: 51
Historical seq: 47
Classification: Anomalous
Retrieved similar logs:
build 44.12 1302.50 480.10 fail 1 2
deploy 57.20 1390.60 240.00 fail 1 0
scan 39.90 5901.10 330.20 pass 0 2
```

Example 3

```
Sequence ID: 914
Historical seq: 913
Classification: Anomalous
Retrieved similar logs:
Run dependency scan
Error: License violation detected
Manual approval pending
Workflow completed with warnings
```

4.5.2 LLM-Enhanced RAG Explainability (RAG+LLM)

Although only RAG explainability is good in offering a strong contextual explanation, retrieved logs can be interpreted manually hence a time-consuming process. The framework is also expanded with a Large Language Model (LLM)-assisted explanation layer to make it more accessible and efficient in terms of operations. The LLM does not perform anomaly detection but instead generates human readable structured explanations using historical logs.

LLM Selection and Justification

Mistral-7B-Instruct is used on the RAG outputs to produce human understandable explanations of the anomalous logs, transforming the historical context into something that can be acted upon and the outputs remain reliable and grounded on the retrieved sequences.

Why this model fits my work:

- Strong instructional and technical logic to interpret correctly retrieved anomalies.
- It can be effectively deployed with low precision to explain the logs of the test phase in real-time.
- Integrating explanations by purpose, to ensure that RAG generates translated working insights correctly.

Integrated Explanation Workflow (LLM on RAG output)

RAG will retrieve semantically close historical logs to give a grounded context to interpretation as in each anomalous log sequence. These recalled logs, as well as sequence metadata, including ID, classification and previous relevant sequences are combined into a formatted prompt to the Mistral-7B-Instruct LLM. The prompt clearly tells the LLM to produce a summary of the issue, causal logic on retrieved historical context and remediation measures that can be taken in the form of a section titled Suggested fix. By generating analyses only found on RAG output that are readable by humans, transparent, and operationally significant, the LLM provides more rapid root-cause analysis and helps to comprehend the DevSecOps test-phase pipelines. As referred to in Figure 4.9, the combined RAG and LLM workflow provides explainability for DevSecOps test-phase logs.

Steps:

- **RAG Retrieval Input:** This provides input as concrete data of historical anomalous sequences to ground explanations and ensure their reproducibility.
- **Prompt Construction:** Takes the logs retrieved, the metadata of the sequences and formatted instructions (including Suggested fix) and uses them to guide the LLM to generate transparency in the output of operations.
- **Explanation Generation:** Converts structured RAG results into short human readable summaries, causal explanation and remedial action taken.
- **Trust & Interpretability:** This makes all information very attached to historical information that can be retrieved when they become much more transparent and free of hallucinating.
- **Operational Advantage:** Suggested fix is a dedicated section that allows the teams to take up immediate action to help reduce the anomalies more effectively and also enhance reliability of the pipelines at the test stage. Any generated explanations are deeply rooted on accessed historical data and interpretability is preserved and chances of hallucination minimized.

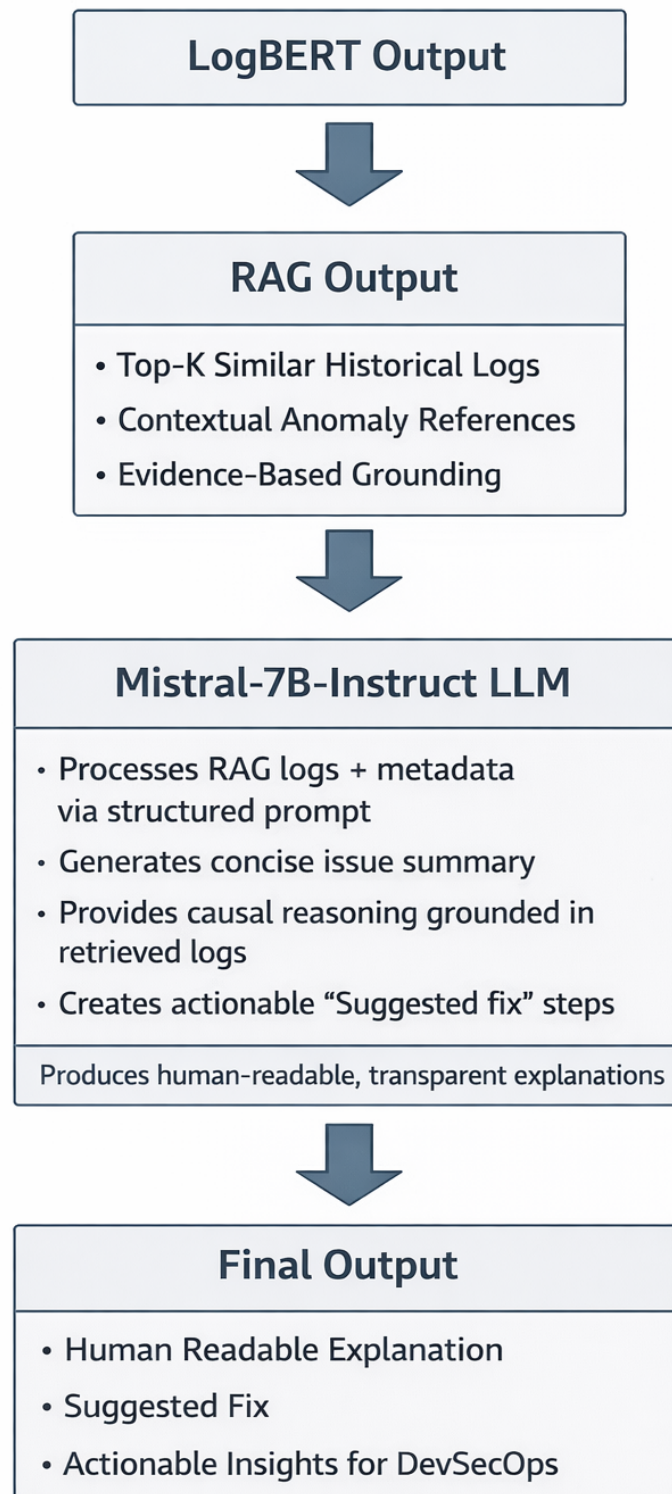


Figure 4.9: RAG + LLM Explainability Workflow in DevSecOps Test Phase

Example (RAG+LLM Output)

Example-1:

Sequence ID: 7994 — Anomalous

Issue: Anomalous sequence with increased number of intrusion attempts and blocked IP addresses.

Moreover, This sequence exhibits a high rate of intrusion attempts, with multiple instances of "Intrusion" detected in the logs. The IP addresses associated with these attempts originate from various locations, including India and Assam. The operating systems identified include Linux and Windows. The X11 and AppleWebKit components also appear frequently. These patterns match those observed in retrieved sequences 6860 and 30989, which occurred approximately 67.47 seconds and 67.65 seconds ago, respectively.

Reason:

Sequence 6860: A previous anomalous sequence featuring similar patterns, including multiple intrusion attempts, IP addresses from India, and the Linux operating system. The high rate of intrusion attempts and blocked IP addresses in this sequence may indicate a reconnaissance phase of the attack or an attempt to identify vulnerabilities in the system.

Sequence 30989: Another anomalous sequence with overlapping patterns, such as intrusion attempts, IP addresses from various locations, and the AppleWebKit component. This sequence occurred shortly before the current sequence and may represent a continuation or escalation of the attack campaign.

Suggested fix:

1. Implement additional security measures, such as firewalls and intrusion detection systems, to protect against DDoS attacks and prevent unauthorized access.
2. Apply software patches and updates to address known vulnerabilities in the Linux and Windows operating systems.
3. Monitor and analyze system logs for any unusual activity or patterns, particularly those related to X11 and AppleWebKit components.
4. Implement rate limiting and traffic shaping to mitigate the impact of potential DDoS attacks.
5. Configure the firewall to block traffic from known malicious IP addresses or regions.

Example-2:

Sequence ID: 51 — Anomalous

Issue: Build and scan times drastically decreased, deploy failed with increased CPU usage.

Moreover, The CPU usage spike during the deploy step indicates potential resource contention or misconfiguration. The significant decrease in build and scan times suggests possible optimization or parallelization, but the failed deploy step is concerning.

Reason: Retrieved Sequence ID 47: Build and scan times were consistently low, but the deploy step failed with high CPU usage and longer duration.

Sequence 47-this sequence exhibited similar behavior to the current sequence, with reduced build and scan times and a failed deploy step. The CPU usage during the deploy step was significantly higher than usual, suggesting a resource issue. The current sequence's anomalous behavior may be attributed to resource contention or misconfiguration during the deploy step. The decrease in build and scan times could be due to optimization or parallelization, but the failed deploy step indicates an underlying issue. Comparing the current sequence with Retrieved Sequence ID 47 reveals similar patterns, as both sequences exhibit reduced build and scan times and a failed deploy step with increased CPU usage.

Suggested fix:

1. Review resource allocation for the deploy process to ensure sufficient CPU and memory availability.
2. Investigate potential misconfigurations in the deploy process, such as incorrect environment variables or dependencies.
3. Implement resource throttling or limiting during the deploy process to prevent resource contention.
4. Monitor the system closely during deployments and address any issues promptly to minimize downtime.

Example-3:

Sequence ID: 914 — Anomalous

Issue: License violation detected during Node.js setup and artifact upload.

Moreover, The current sequence's anomalous behavior is likely due to a persistent licensing issue within our Node.js setup process. This issue was previously observed in sequence 913, where the same error message was encountered during the license check step. The high error rate in both sequences suggests that there may be a problem with our Node.js installation or the licensing server.

Reason: Historical sequence 913: Sequence 913 also experienced a license violation error during the Node.js setup process. The error occurred at the same step as in sequence 914, indicating a recurring issue. This sequence did not result in any significant deviations from normal behavior in terms of CPU usage, memory consumption, or duration. However, the error rate was high, making it relevant to the current sequence's anomaly.

Suggested fix:

1. Review Node.js installation configuration and ensure proper licensing information is provided.
2. Contact licensing vendors for assistance with the license conflict.
3. Implement a retry mechanism for license checks during Node.js setup.
4. Monitor license checks closely and investigate any future occurrences.

4.5.3 Practical DevSecOps Benefits (RAG + LLM)

The effective integration of RAG with a LLM can complement the activity of test-phase anomaly detection with the help of historical background and readable instructions.

Key Benefits:

- **Contextual Anomaly Understanding:** RAG accesses historical anomalous logs, and the LLM perceives them, assisting teams to connect the current anomalies with the historical pattern.
- **Human-Readable Explanations:** The LLM replaces raw sequences with structured and summaries that have cause-and-effect explanations and recommendations.
- **Actionable Remediation Guidance:** The LLM provides the corrective actions based on the historical log patterns and enables the prompt mitigation.
- **Operational Transparency and Trust:** Historically-based explanations of LLM make the automated detection more understandable and more trustworthy.
- **Pattern and Trend Awareness:** By comparison of the anomalies with the other sequences using the LLM, teams can determine events of recurring failures or inefficient workflow, and can preventively make improvements.

The structure produces interpretable, contextual and practical observations, by combining the RAG retrieval and synthesis capabilities provided by LLM, and simplifies the task of resolving anomalies and enhancing operations, when conducting the test phase of DevSecOps.

Chapter 5

Performance Evaluation and Analysis

5.1 Performance Evaluation

This part assesses the functionality and accuracy of the suggested LogBERT-based anomaly detection system through the examination of its ability to detect anomalous sequence of DevSecOps pipelines in a reliable way. The evaluations of performance are determined by the consistency of explanation and not raw scores of prediction, and the key issues to consider here are whether the identified anomalies generate consistent, understandable, and practical explanations.

There are two complementary mechanisms of explanation RAG+LLM and SHAP+LLM that are evaluated on the same anomalous sequences. Because both techniques are motivated by LogBERT outputs, the convergence between them implies their consistency in the explanations in the case of the DevSecOps test phase.

5.2 Validation of LogBERT Anomaly Detection

The accuracy of the LogBERT model has been proven by comparing the explanations obtained through RAG+LLM and SHAP+LLM on the chosen anomalous sequences. As both modes of explanation are based on the LogBERT outputs, RAG means that data is historically-grounded, and SHAP means that tokens contribute to the overall effect, it means that the consistency between the two methods shows that LogBERT is a dependable anomaly detector. Table 5.1 illustrates the representative sequences with both methods finding the same root causes indicating that LogBERT also generates accurate and practicable anomaly predictions.

Sequence	RAG+LLM	SHAP+LLM	Conclusion
7994	High intrusion rate, blocked IPs, patterns similar to previous attacks	Repetition of DNS/Intrusion tokens (Repudiate, eligendi, etc.)	Both point to coordinated attacks / abnormal traffic
51	CPU spikes, failed deploy, decreased build/scan times	Fail/build/scan/deploy tokens with high anomaly scores	Both indicate resource contention / failed build processes
914	License violation during Node.js setup	Dependency/license-related tokens (dependency, License, violation)	Both indicate license violation / setup error

Table 5.1: LogBERT Anomaly Detection Validation (RAG vs SHAP)

This kind of comparison will show that the results of the LogBERT on the problem of anomalies are consistent and trustworthy, since context-based (RAG) and token-based (SHAP) descriptions are directed towards the same interpretations.

5.3 Analysis of Design Solutions

The design combines the context-driven and the token-driven explanation strategies to authenticate the predictions of anomalies by LogBERT

Why Use Both RAG and SHAP

- **Not only RAG:** At the DevSecOps phase of testing, using RAG to retrieve log sequence history gives a helpful contextual base to interpreting anomalous events. However, RAG alone does not indicate what exact tokens, log events, or operations are causing anomalies, and only offers some actionable information of what is happening at the fine-grained sequence level.
- **Not only SHAP:** SHAP adds to this by offering token-level explanations of the anomalies that are most useful to predict the anomaly score of LogBERT, including what character or operation is the most useful to overall anomaly scores. Although this provides more localized and accurate information, SHAP is unable to attribute anomalies to historical patterns or the context of operation, which makes it challenging to observe patterns and evaluate a wider impact on the pipeline by the team.
- **Combined Approach:** The framework produces a dual-explanation viewpoint by incorporating the historical context of RAG and token-level evidence of SHAP and, as a result, provides a better anomaly detection view in the DevSecOps test phase. Such an integrated solution will help the teams identify the precise roots of anomalies, comprehend their importance against the

background of historical events, and take specific corrective actions with minimal latency. It makes development and security teams reduce pipeline downtime, proactively identify recurring anomalies and maintain consistent CI/CD pipeline operation, making it easier to interpret, operationally visible, and make a decision.

A combination of the two approaches to explanation improves the operational decision making, gives traceable actionable information, and speeds up the root-cause analysis. In the case of DevSecOps pipelines, this methodology assists in improving system reliability, security validation and enforcement of compliance.

5.4 Comparisons and Relationships

To confirm the reliability and interpretability of the LogBERT anomaly detection, we compare the explanation of RAG + LLM and SHAP + LLM to specific sequences. Unlike RAG+LLM which is grounded in the historical context, where current anomalies are shown relative to historical sequences, SHAP+LLM is someone that can point to the contribution of tokens to determine specific operational or security issues. The combination of the two approaches is necessary to guarantee higher accuracy, transparency, and actionable insights as it is extremely important during the DevSecOps test phase to perform proactive validation, targeted troubleshooting, and compliance verification. In this section of the paper, the relationships and efficiency of the two approaches will be compared and contrasted to confirm their connection.

Sequence-Level Observations

Sequence ID-7994	
Only RAG+LLM	Only SHAP+LLM
Shows anomalous intrusion attempts, blocked IPs, historical context from sequences 6860 & 30989	Shows repeated DNS tokens (Repudiate, eligendi, temporibus, etc.) indicating DNS loops/amplification
Both (RAG + SHAP): High intrusion attempts with repeated DNS/intrusion tokens (Repudiandae, eligendi, temporibus) match patterns observed in historical sequences 6860 and 30989.	
DevSecOps Test Phase Benefit: These patterns can be used to determine that they are dealing with a coordinated attack campaign and what IPs, OS, and components are compromised to focus mitigation at the earliest stage during testing, which can proactively configure firewall/IDSs.	

Table 5.2: SHAP vs RAG Anomaly Analysis – Example 1

Sequence ID-51	
Only RAG+LLM	Only SHAP+LLM
Shows CPU spikes, failed deploy, decreased build/scan times, compares with previous sequences	Shows token-level failures: fail, scan, build, deploy, UNK
Both (RAG + SHAP): CPU spikes, failed deploy, and decreased build/scan times match patterns from previous sequences with similar resource issues. Tokens fail, scan, build, deploy pinpoint the exact failing steps.	
DevSecOps Test Phase Benefit: Performance and failure pattern matching supported by performance and deployment troubleshooting. Compare performance and failure patterns to target deploy steps and resource allocation, enhancing system stability.	

Table 5.3: SHAP vs RAG Anomaly Analysis – Example 2

Sequence ID-914	
Only RAG+LLM	Only SHAP+LLM
Shows license violation during Node.js setup, references previous sequence 913	Shows license-related tokens: dependency, packages, License, violation
Both (RAG + SHAP): License violation during Node.js setup and dependency scan matches the recurring pattern seen in sequence 913. Tokens dependency, packages, License, violation mark the exact failure points.	
DevSecOps Test Phase Benefit: Identifying recurring licensing problems with token-level evidence is useful in instructing corrective actions and enforcing compliance throughout the Node.js configurations. Ensures compliance checking at the test level before downstream build breakage and maintains adherence to DevSecOps policies.	

Table 5.4: SHAP vs RAG Anomaly Analysis – Example 3

5.5 Final Design Adjustment

The final design, which is chosen on the basis of the comparative analysis, is a formal combination of RAG+LLM and SHAP+LLM explanation strategy, which is used as an internal validation layer of the LogBERT-based anomaly detection. The rationale behind this modification is seen by the uniform correlation found between historical context explanations and token level attributions between anomalous test-phase sequences.

Anomalies should not only be detected in the right way but also in a manner that leads to quick root cause analysis and corrective actions in DevSecOps testing environments. RAG+LLM makes validation stronger because anomalies are based on the historical behavior of pipelines and SHAP+LLM offers accurate evidence of the most contributing log tokens and operations to abnormality. Tables 5.2, 5.3, and 5.4 show SHAP vs RAG anomaly analysis for Examples 1, 2, and 3, respectively. The combination of the two assures that the outputs of LogBERT can be interpreted, replicated and taken into action.

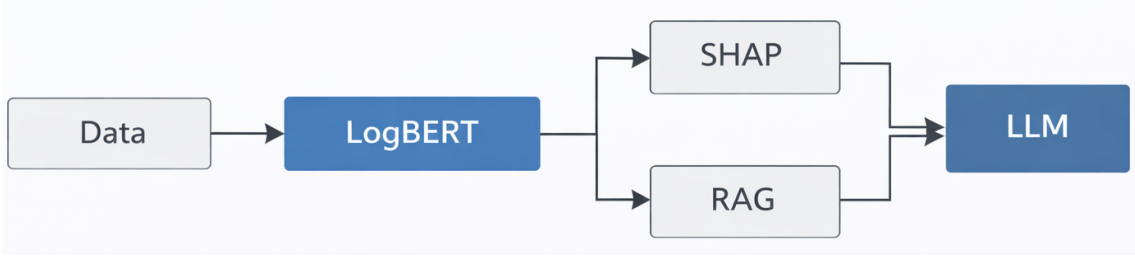


Figure 5.1: End-to-end workflow of the system

This last design change (As shown in Figure 5.1) enhances the efficiency of root-cause analysis at the test stage and makes it possible to mitigate the situation in advance before its launch. The framework, in turn, enhances the stability of DevSecOps pipelines and reduces risks related to security and compliance as well as supports successful DevSecOps decision-making.

5.6 Discussion

The findings indicate that SHAP+LLM and RAG+LLM explanations are more effective, easier to interpret, and operationally useful at explaining the LogBERT-based anomaly analysis of DevSecOps test-phase pipelines. Context-based explanations made with RAG provide historical information but do not give detailed information about the behavior of the mode whereas token-level explanations made with SHAP show particular deviations but do not give the operational importance of the deviations. With the combination of these strategies, the framework supports the gap between what failed (token-level evidence) and why it matters (historical and operational context) so that development and security teams can be able to know the anomalies in a holistic manner and react to them with confidence. Table 5.5 highlights the comparison between conventional and proposed DevSecOps anomaly detection approaches.

Addressing RQ1 (Anomaly behaviour Detection)

LogBERT models normal DevSecOps execution patterns and identifies anomalies by detecting deviations in log sequences. The framework can point out unexpected behaviors in a pipeline by its capability to define orderly and contextual connections among log entries. Accordingly, the RQ1 is answered through offering an effective system to identify abnormal execution behavior in real-time test-phase logs.

Addressing RQ2 (Unified Token-Level and Contextual Explainability)

The integration of SHAP and RAG creates a dual explanation mechanism. SHAP measures the relevance of each log token to an anomaly, and RAG replicates semantically related sequences of historical logs to give more context to operations. Together, they offer explanations that encompass both local root causes and the global operational significance of deviations, directly addressing RQ2 by producing clear, interpretable insights into anomalous behavior.

Addressing RQ3 (Evidence-Based Explanation and Generation of Actions)

The LLM-based reasoning layer synthesizes token-level attributions and historical evidence to generate human-readable explanations and practical remediation advice. This will allow developers and security engineers to learn more about the cause of anomalies, how they affect operations, and take corrective action. RQ3 is therefore resolved by the provision of practical evidence-based information that can enable timely and informed responses within DevSecOps pipelines.

Overall, the framework improves transparency in automated anomaly detection, reduces ambiguity in anomaly alerts, and strengthens security and compliance. The fact that the token-level and contextual accounts are combined also guarantees that the LogBERT-based framework is reliable, understandable, and operationally applicable, and it is especially applicable to the test-phase DevSecOps pipelines, where the rapidity of anomaly detection, interpretation, and action is the most important.

Aspect	Conventional Approaches	Proposed Framework
Anomaly Detection Methodology	Predominantly rule-based or supervised models relying on predefined patterns or labeled datasets	Self-supervised LogBERT learns normal log sequence behavior; anomalies are detected as deviations from learned patterns
Model Interpretability	Limited interpretability; anomaly alerts are often black-box outputs	Provides token-level explainability via SHAP, enabling precise root-cause identification
Contextual Awareness	Analysis restricted to individual log sequences with limited historical understanding	RAG retrieves semantically similar historical log sequences to provide contextual insights
Explanation Generation	Manual investigation required to interpret alerts and infer causes	LLM reasoning layer combines SHAP attributions and retrieved context into human-readable explanations
Operational Actionability	Primarily focused on anomaly detection with minimal guidance for response	Generates evidence-based remediation guidance suitable for DevSecOps test-phase operations

Table 5.5: Conventional vs Proposed DevSecOps Anomaly Detection

Chapter 6

Conclusion

6.1 Summary of Findings

This study explored how explainable artificial intelligence can be applied to observe anomaly behavior during the test-phase of DevSecOps. Functional testing, security scanning, deployment, and monitoring pipelines produce large amounts of heterogeneous logs generated by Continuous Integration and Continuous Deployment (CI/CD) pipelines. Traditional rule-based and black-box anomaly detection systems are not always transparent, which in turn leads to low trust and slow incident response.

To resolve these limitations, this research proposed an explainable anomaly detection model that combines **LogBERT** with **SHAP**-based interpretability and **Retrieval-Augmented Generation (RAG)**. Transformer-based LogBERT was trained to learn normal execution patterns using unlabeled DevSecOps logs and estimate the likelihood of reconstructing tokens to suggest abnormal log behavior. Experimental results demonstrate that LogBERT effectively captures sequential dependencies in multi-source test-phase logs and detects previously unobserved behaviors without supervised labels.

SHAP was applied to provide token-level attribution of anomalies, allowing accurate identification of the log elements responsible for abnormal behavior. This significantly improved interpretability, as anomaly alerts were transformed into traceable evidence. Additionally, the RAG mechanism retrieved semantically similar historical log sequences, offering a contextual basis for anomaly analysis. These outputs were further processed by a large language model (LLM) to generate structured, human-readable explanations and suggested remediation actions.

The findings indicate that the combination of transformer-based anomaly detection, explainable reasoning, and retrieval-based contextual grounding provides credible, transparent, and operationally meaningful insights into DevSecOps test-phase pipelines.

6.2 Contributions to the Field

This research makes the following contributions:

- **Explainable DevSecOps Anomaly Detection Architecture:** A consistent structure that incorporates anomaly assessment, interpretability, contextual recovery, and explanation generation in DevSecOps pipelines.
- **Extension of LogBERT to DevSecOps Logs:** Demonstrated the effectiveness of LogBERT in analyzing heterogeneous non-system logs from DevSecOps tests.
- **Token-Level Interpretability Using SHAP:** Enabled high-resolution attribution of anomaly-driving tokens to support root-cause analysis rather than general alerting.
- **Context-Grounded Explanation via RAG:** Automatically retrieved semantically similar historical traces to improve reproducibility, auditability, and operational context.
- **LLM-Assisted Explanation Generation:** Translated technical evidence into structured, actionable explanations and proposed remediation actions.
- **Support for Trustworthy AI in CI/CD Pipelines:** In-built explainability and evidence-based reasoning promote responsible use of AI-driven automation.

6.3 Recommendations for Future Work

Potential extensions to further enhance the framework include:

- **Online and Incremental Learning:** Introduce adaptive learning to support changing pipeline configurations and software updates.
- **Graph-Based Dependency Modeling:** Model dependencies among pipeline stages, services, and infrastructure to enhance multi-stage anomaly correlation.
- **Automated Response and Self-Healing Pipelines:** Investigate controlled remediation plans based on approved explanations.

6.4 Final Remark

This thesis demonstrates that explainable and contextualized anomaly analysis significantly enhances transparency, trust, and operational efficiency in the test-phase observation of DevSecOps. The proposed framework, integrating transformer-based modeling, token-level interpretability, retrieval-based grounding, and structured explanation generation, contributes to the responsible and reliable use of artificial intelligence in modern software delivery pipelines.

Bibliography

- [1] A. K. Sandu, “Devsecops: Integrating security into the devops lifecycle for enhanced resilience,” *Technology and Management Review*, vol. 6, no. 1, pp. 1–18, Feb. 2021, This article discusses integrating security into the DevOps lifecycle to improve resilience.
- [2] M. Cankar et al., “Security in devsecops: Applying tools and machine learning to verification and monitoring steps,” in *Companion of the 2023 ACM/SPEC International Conference on Performance Engineering (ICPE ’23 Companion)*, This work is licensed under a Creative Commons Attribution International 4.0 License., Coimbra, Portugal: ACM, Apr. 2023, pp. 1–5, ISBN: 979-8-4007-0072-9/23/04. DOI: 10.1145/3578245.3584943.
- [3] R. Dwivedi et al., “Explainable ai (xai): Core ideas, techniques and solutions,” *ACM Computing Surveys*, vol. 55, no. 9, p. 835, Sep. 2023, DOI: 10.1145/3561048.
- [4] S. M. K. S. Naveen Pakalapati Jawaharbabu Jeyaraman, “Building resilient systems: Leveraging ai/ml within devsecops frameworks,” *Journal of Knowledge Learning and Science Technology*, vol. 2, no. 2, pp. 230–245, Oct. 2023, Received: 01/10/2023, Accepted: 03/10/2023, Online: 12/10/2023, Published: 12/10/2023. DOI: 10.60087/jklst.vol2.n2.p230.
- [5] J. B. Adelusi and S. Coleman, “Ai-enhanced devsecops: Improving security in devops pipelines,” *ResearchGate*, Sep. 2024.
- [6] N. G. Camacho, “Unlocking the potential of ai/ml in devsecops: Effective strategies and optimal practices,” *Journal of Artificial Intelligence General Science*, vol. 3, no. 1, pp. 106–115, Jan. 2024, Open Access under Creative Commons Attribution 4.0 International License, ISSN: 3006-4023. [Online]. Available: <http://jaigs.org>.
- [7] C. Feio, N. Santos, N. Escravana, and B. Pacheco, “An empirical study of devsecops focused on continuous security testing,” in *2024 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, Under license to IEEE, Lisbon, Portugal: IEEE, 2024. DOI: 10.1109/EuroSPW61312.2024.00074. [Online]. Available: <https://www.inov.pt/en/index.html>.
- [8] M. Fu, C. Tantithamthavorn, T. Le, V. Nguyen, and D. Phung, “Ai for devsecops: A landscape and future opportunities,” in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering (ICSE)*, Publication date: September 2024, IEEE, 2024, pp. 47–56.
- [9] A. Hussain and G. Shabir, “Ai-powered devsecops: Elevating security practices with machine learning,” *ResearchGate*, 2024, Available at: <https://www.researchgate.net/publication/380270996>. DOI: 10.13140/RG.2.2.25673.76640.

- [10] K. Jane, “Aiops-enabled devsecops: Integrating security automation and ai-based threat intelligence across the software delivery lifecycle,” *Journal Name (if applicable)*, Jul. 2024, This paper explores the integration of AIOps in DevSecOps to automate security and improve threat detection.
- [11] T. Olasehinde, T. Gen, and D. Esther, “Real-time monitoring and root cause analysis in ai-driven devops,” *Unspecified Journal*, Feb. 2024, Accessed via ResearchGate on 04 February 2025.
- [12] K. Pandya, “Automated software compliance using smart contracts and large language models in continuous integration and continuous deployment with devsecops,” A thesis presented in partial fulfillment of the requirements for the degree Master of Science, Master’s Thesis, Arizona State University, Tempe, Arizona, Jun. 2024.
- [13] A. K. Polinati, “Devops and ai: Automating software delivery pipelines for continuous integration and deployment,” *Nanotechnology Perceptions*, vol. 20, no. 4, pp. 788–805, Aug. 2024, Published in August 2024, ISSN 1660-6795.
- [14] M. N. K. Yaddala and Y. R. Sunkara, “Comprehensive survey of web security threats in 2024,” *International Journal of Scientific Research in Engineering and Management (IJSREM)*, vol. 8, no. 11, pp. 1–18, Nov. 2024, DOI: 10.55041/IJSREM38614.
- [15] F. Binbeshr and M. Imam, “Comparative analysis of ai-driven security approaches in devsecops: Challenges, solutions, and future directions,” in *Proceedings of The 29th International Conference on Evaluation and Assessment in Software Engineering (EASE 2025)*, 17–20 June, 2025, Istanbul, Türkiye, New York, NY, USA: ACM, Jun. 2025, p. 10.
- [16] L. J. Charlotte, “Machine learning in cloud native devsecops automation,” *ResearchGate*, Mar. 2025, Published online. [Online]. Available: <https://www.researchgate.net/publication/390149877>.
- [17] A. Rahman and M. B. Biplob, “Secureai-flow: A security-oriented ci/cd framework for ai software,” *Preprints.org*, Jun. 2025. [Online]. Available: <https://doi.org/10.20944/preprints202506.0035.v1>.
- [18] P. P. Ray, “A review on vibe coding: Fundamentals, state-of-the-art, challenges and future directions,” *arXiv preprint*, May 2025, Preliminary report on TechRxiv. Available at <https://doi.org/10.36227/techrxiv.1746814>. [Online]. Available: <https://doi.org/10.36227/techrxiv.1746814>.
- [19] M. Song and S. Baker, “Ai-based threat detection and mitigation in devsecops,” *Journal of Cybersecurity and Artificial Intelligence*, Feb. 2025.